

Overview of designing services and system operations

Chris Richardson

Founder of Eventuate.io

Founder of the original CloudFoundry.com

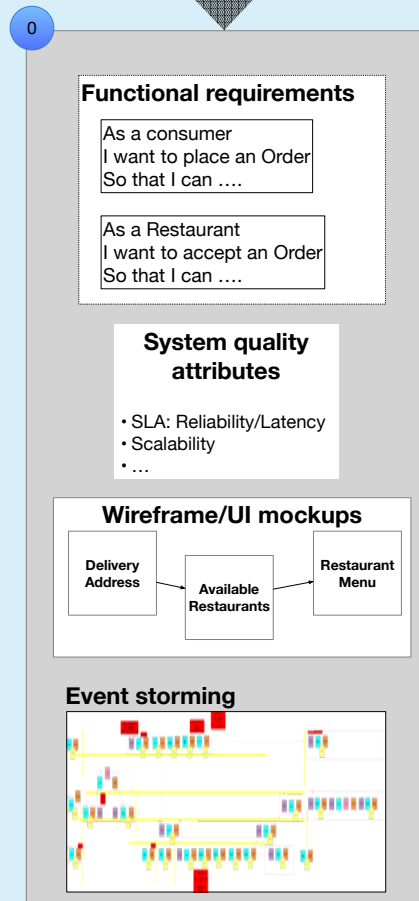
Author of POJOs in Action and Microservices Patterns

 @crichardson

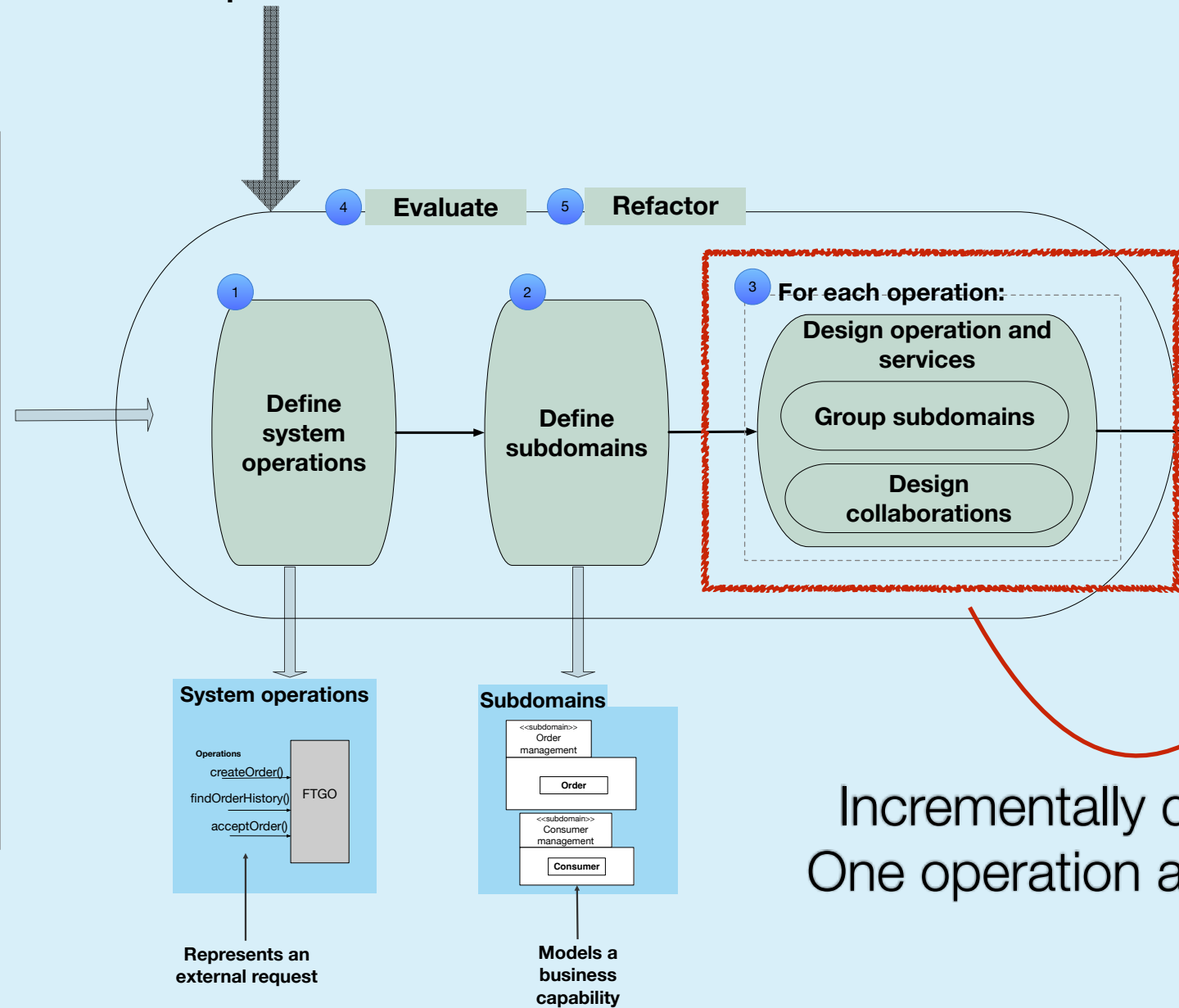
chris@chrisrichardson.net

adopt.microservices.io

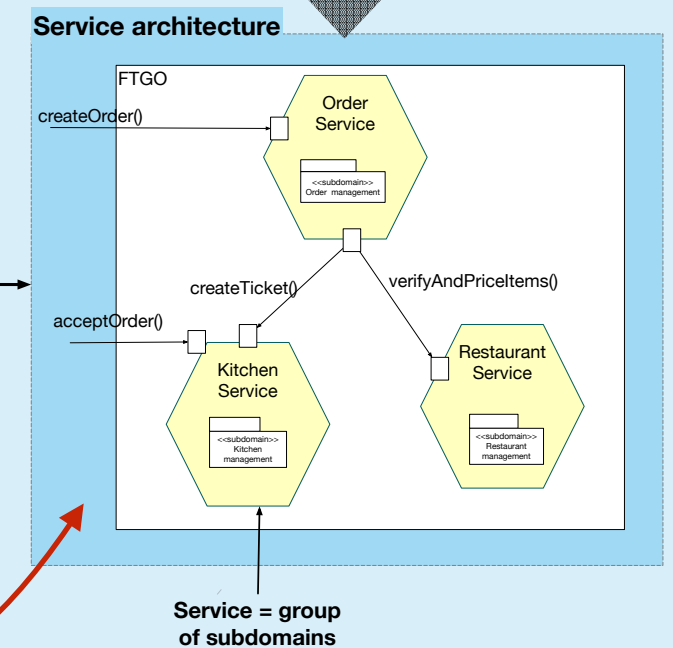
The starting point



The process



The result

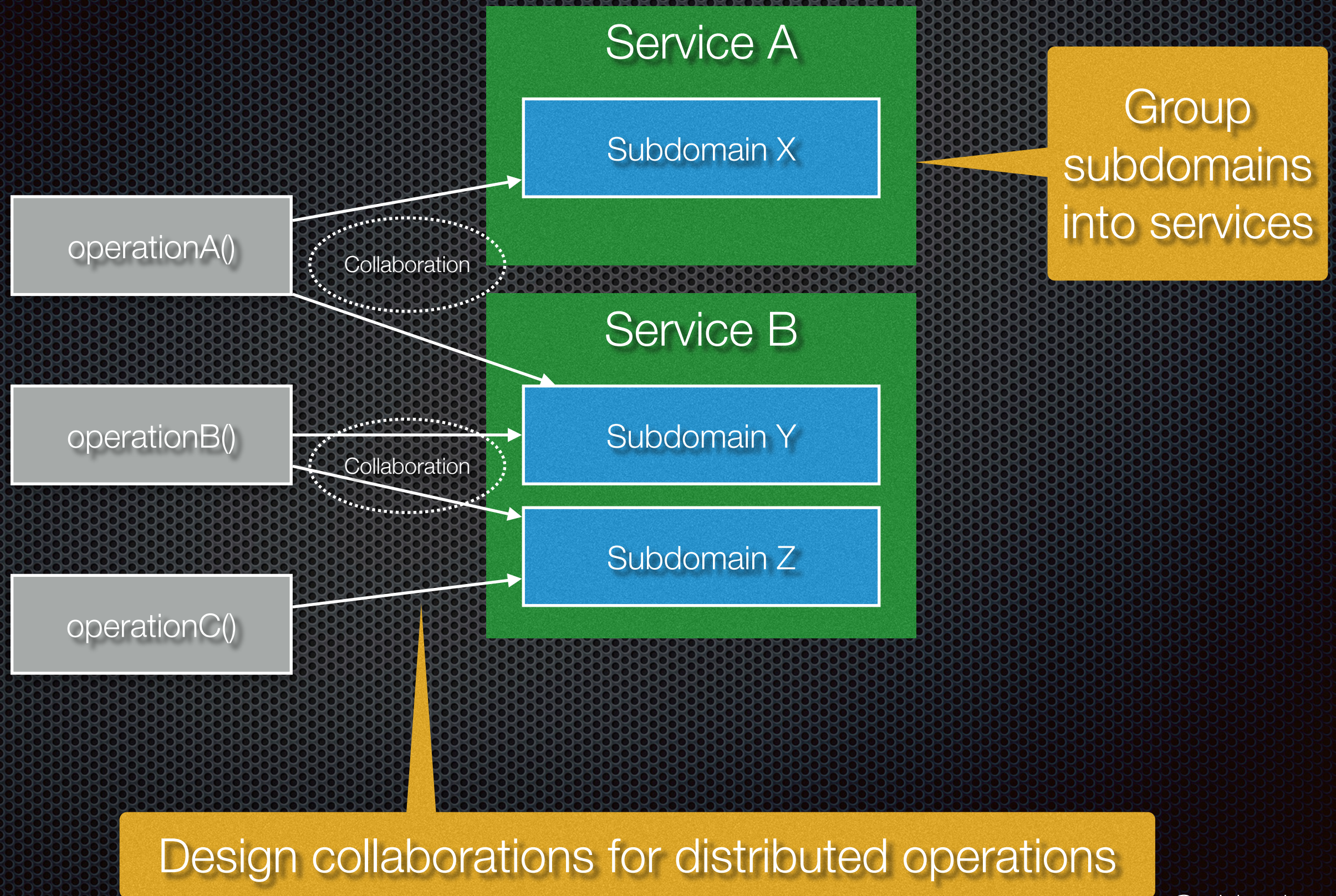


Incrementally defines
One operation at a time

Agenda

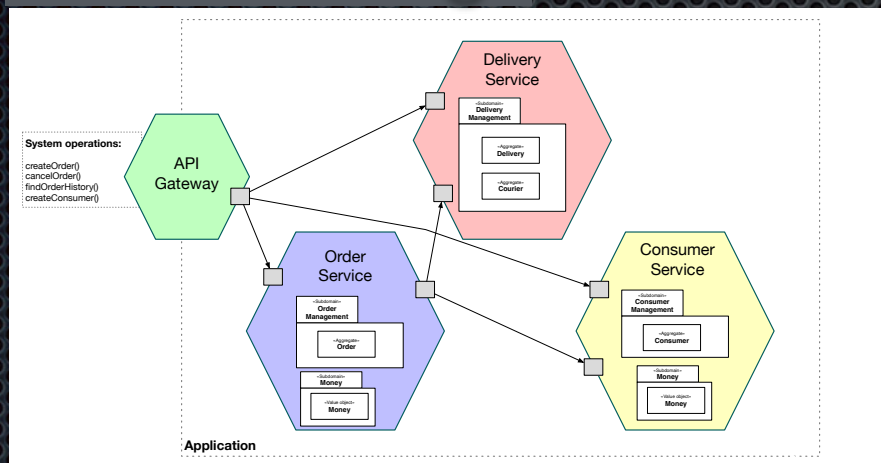
- Overview of designing services and system operations
- Documenting services and collaborations
- Ranking operations
- Generating candidate service and operation designs
- Evaluating candidate designs

Problem: defining services



Designing an architecture = making trade-offs

Service design



Constrains

- Simplicity
- Efficiency
- Runtime coupling
- Prefer ACID over BASE

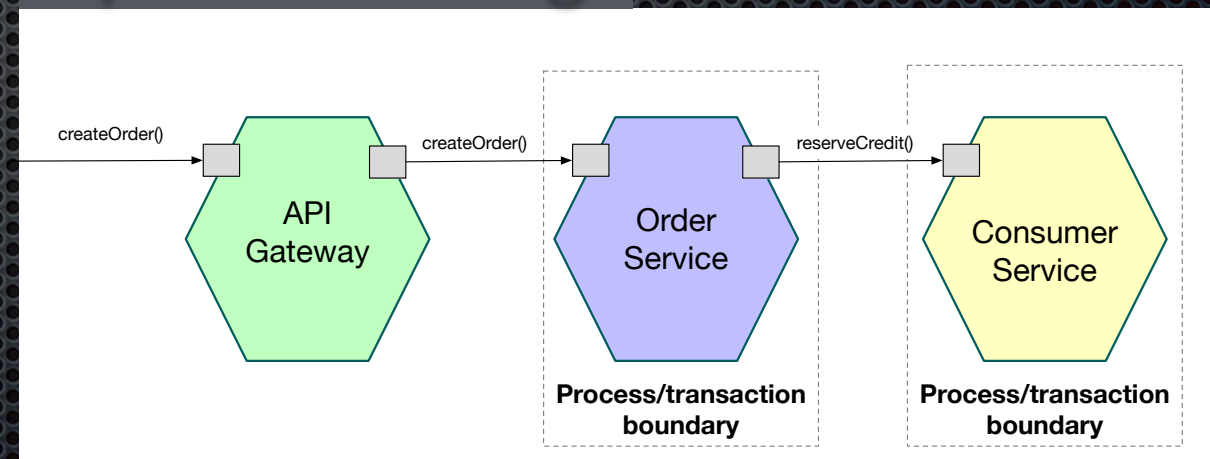
Conflict

Determines

Determines

Determines

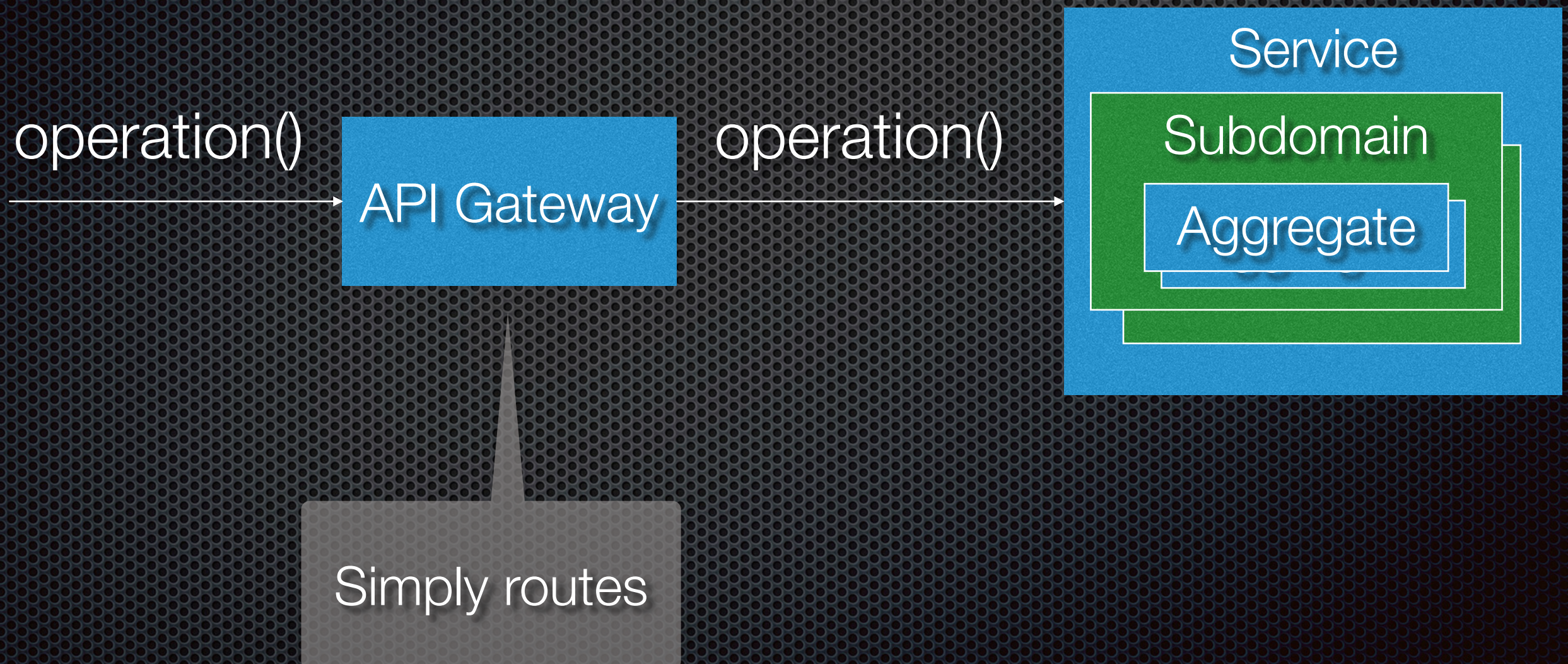
Operation design



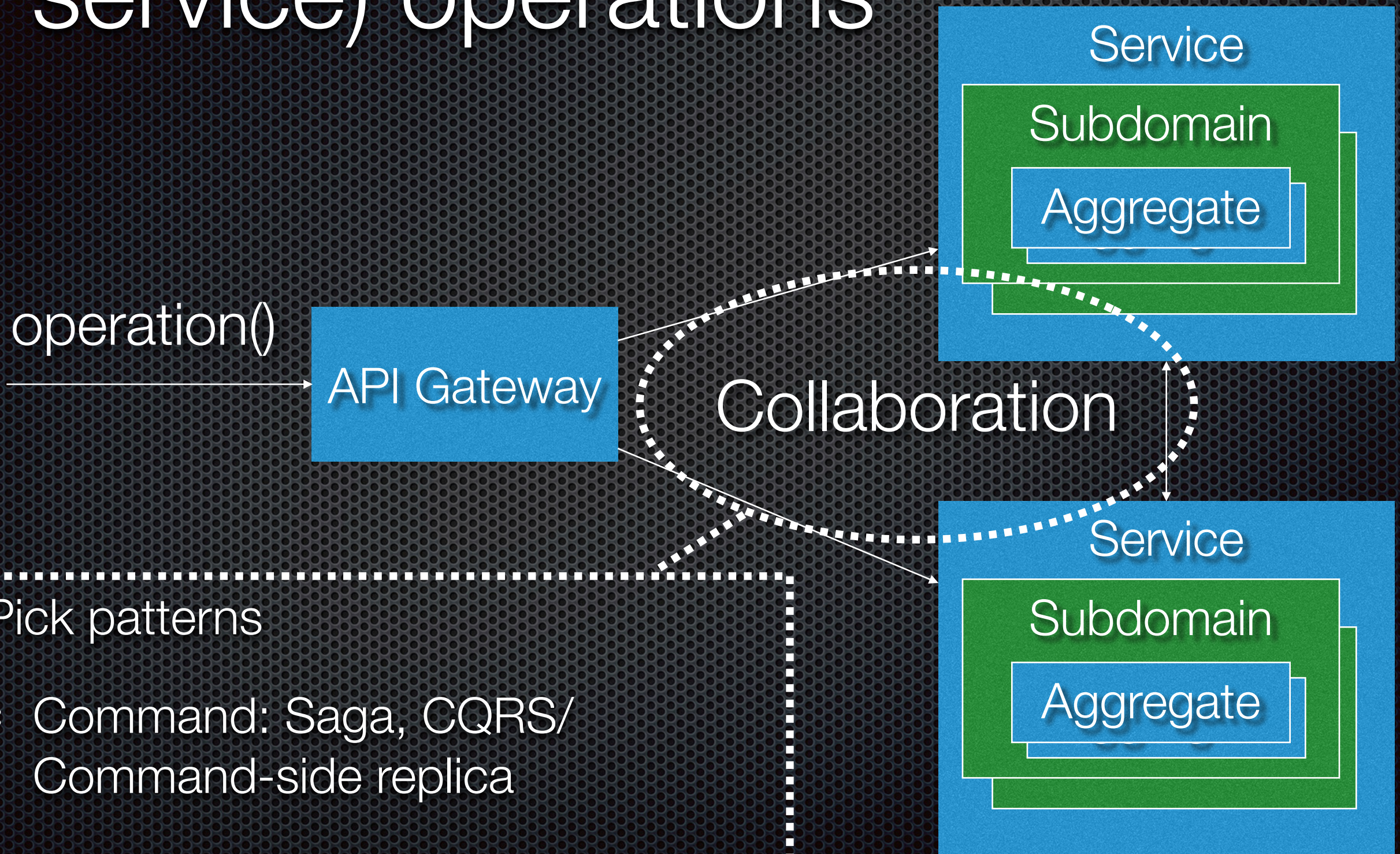
- Loose-coupling
- Testability
- Deployability

Requirements for fast flow

Designing single service operations = easy



Designing distributed (multi-service) operations



Designing services and system operations

3

Design services and operations

Rank system operations

For each operation:

Generate candidate designs

Group subdomains

Design collaborations

Evaluate candidates

Select candidate design

System operation specifications

Command specification

Command Canvas

Query specification

Query Canvas

Subdomain definitions

Subdomain canvas	
Name	Restaurant Management
Responsibilities	- Manages information about restaurants - Knows restaurant name, location, description, opening hours, and menu
Collaborations	External address validation and geocoding service
Type	Supporting
Complexity	Simple
Key aggregates	Restaurant

Service architecture

Service definitions

Service

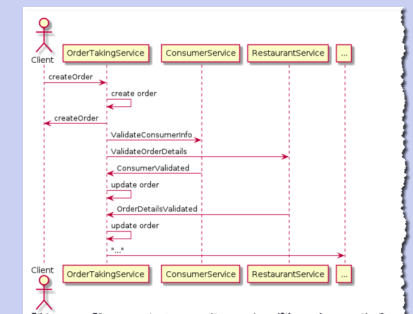
Subdomain A

Subdomain B

Operation collaboration design

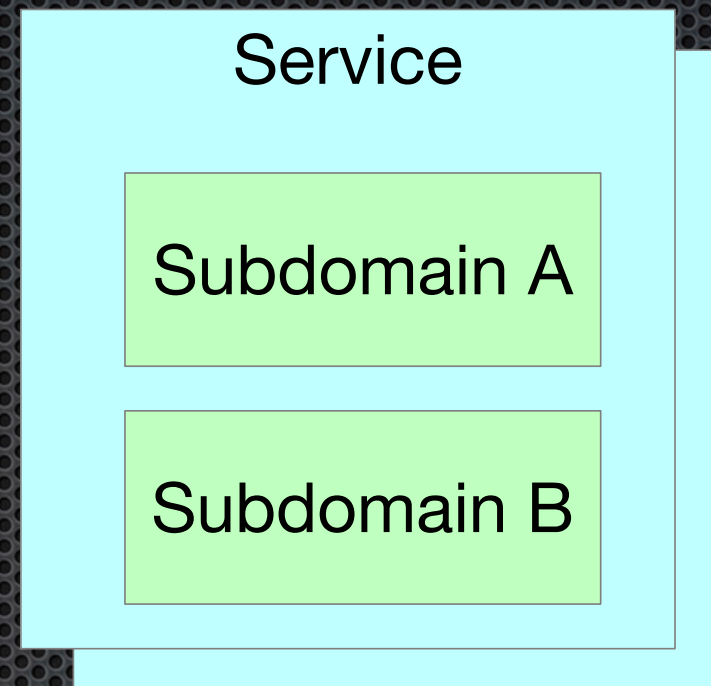
System command:	createOrder()		
Initiating service:	Order Service		
Steps			
Step No.	Participant	Transaction	Compensating Transaction
1	Order Service	createOrder()	rejectOrder()
2	Customer Service	reserveCredit()	-
3	Order Service	approveOrder()	-

Query:	findOrder()
Parameters:	TBD
Returns:	TBD
Strategy:	API Composition
Participants	
Order Service	findOrder()
Delivery Service	findDeliveryByOrderid(orderid)
Kitchen Service	findTakeByOrderid(orderid)



Deliverables: service definitions

- ✦ Name
- ✦ Responsibilities
- ✦ API
- ✦ Subdomains
- ✦ Collaborations

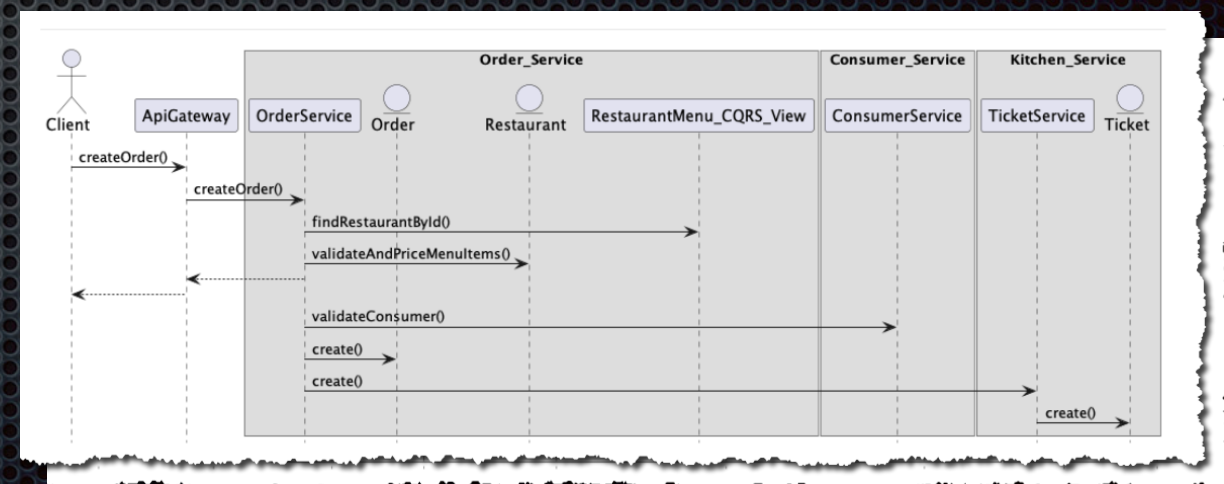


Service canvas		
Name:	Order Service	
Description:		
Responsibilities	• Manages orders	
Subdomains	• Order management	
Service API		
Commands	Queries	Events Published
• createOrder()	• findOrder()	• OrderCreated
Non-functional requirements		
Dependencies		
Invokes	Subscribes to	
	Restaurant Service: • Restaurant Created • Restaurant Updated • Restaurant ...	

Deliverables: system operation designs

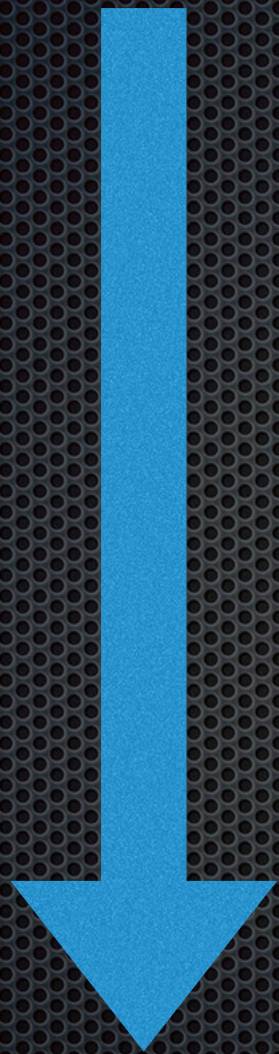
- ✦ Operation entry point = service that (first) handles operation
- ✦ If distributed THEN collaboration patterns

System command design canvas	
Name:	createOrder()
Entry point:	
Service:	Order Service
Operation:	createOrder()
Collaboration patterns:	
Sagas	• Create Order
Command-side Replicas	• Restaurant Menu View



Step #1: Rank operations

Most important

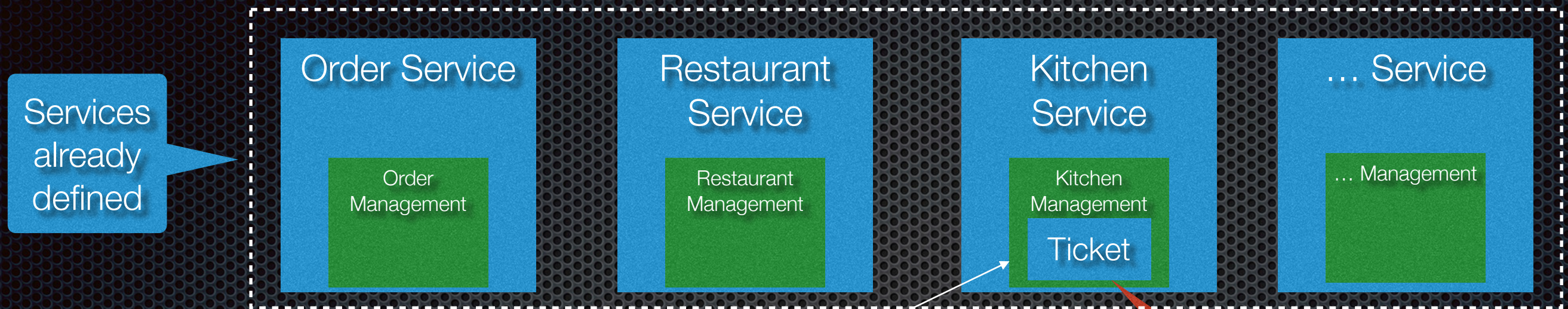


createOrder()
acceptTicket()
...()

Least important

Step #2: For each system
operation

Step #2a: Group its subdomains



Ranked operations

`createOrder()`

`acceptTicket(ticketID)`

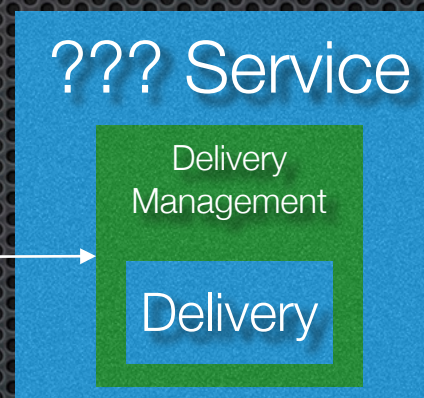
Pre-conditions:
Exists **Ticket** `t`
where `t.ID = ticketID`
and `t.state = PENDING`

Post-conditions:
`t.state` became `ACCEPTED`
Delivery `d` was created
`d.state` became `SCHEDULED`
`d...` Became ...

`noteAccepted()`

Already assigned

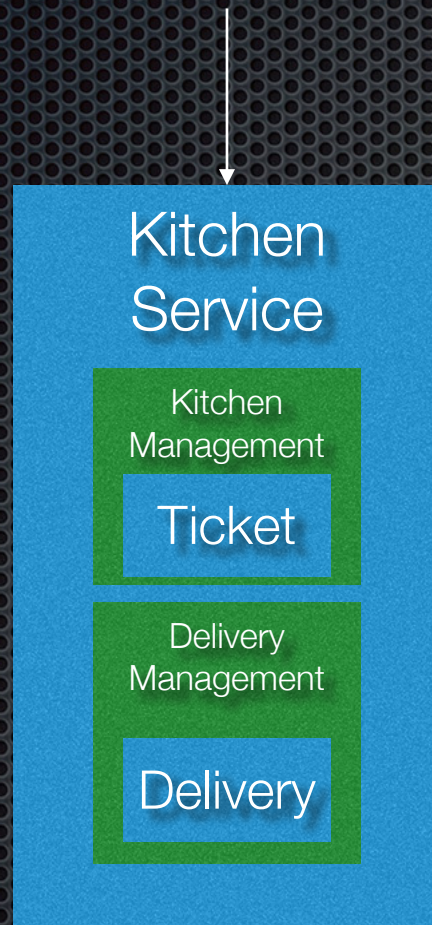
`scheduleDelivery()`



Must assign to service

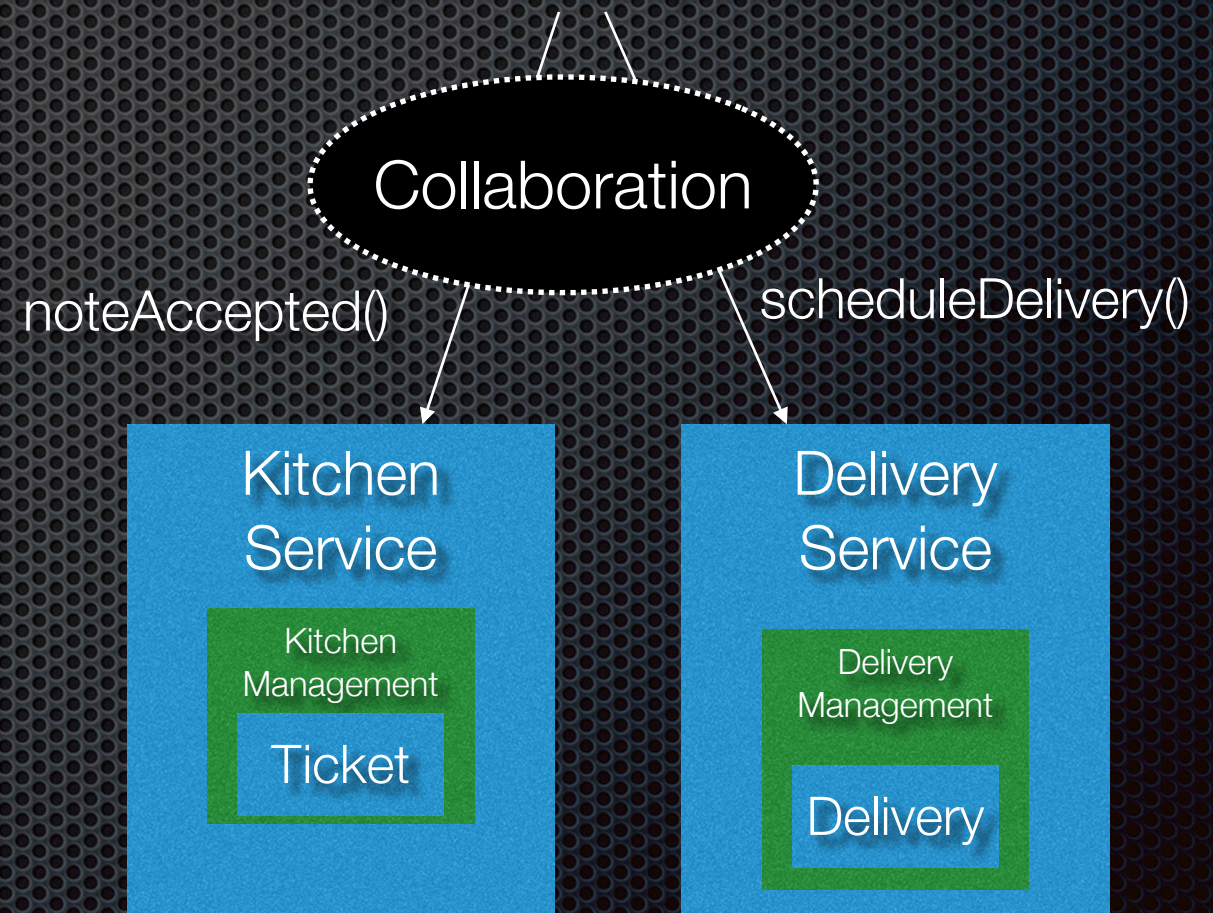
Step #2b: if operation is distributed THEN design collaborations

acceptTicket(ticketID)



Local operation

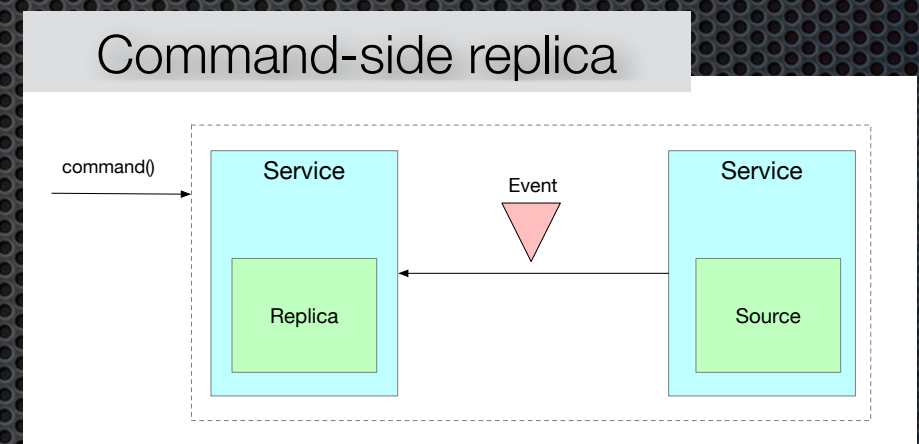
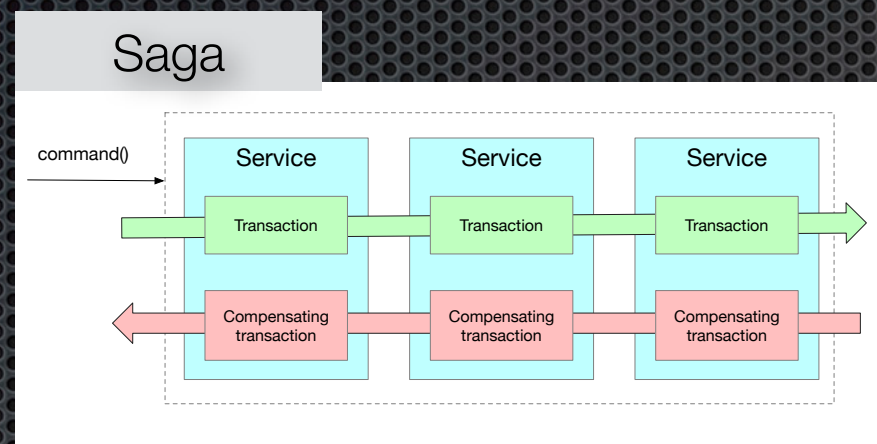
acceptTicket(ticketID)



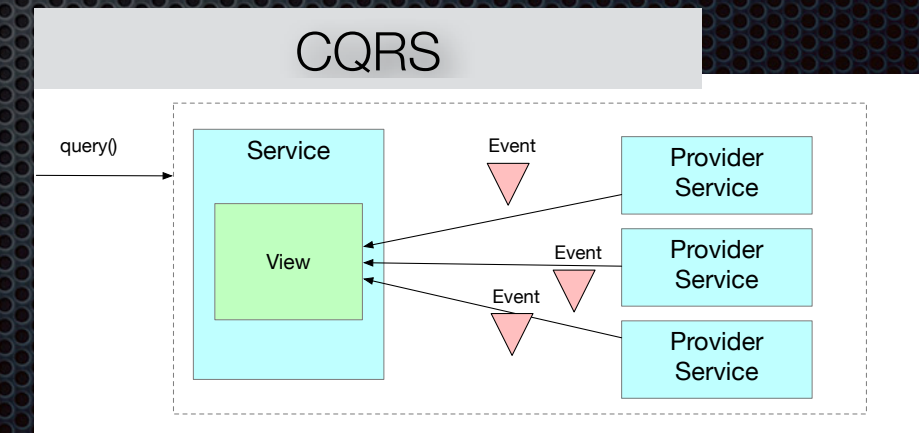
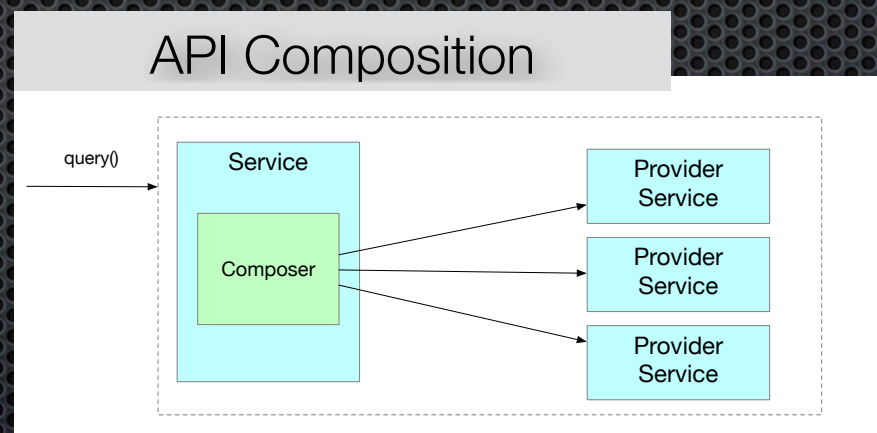
Distributed operation

Service collaboration patterns

Commands

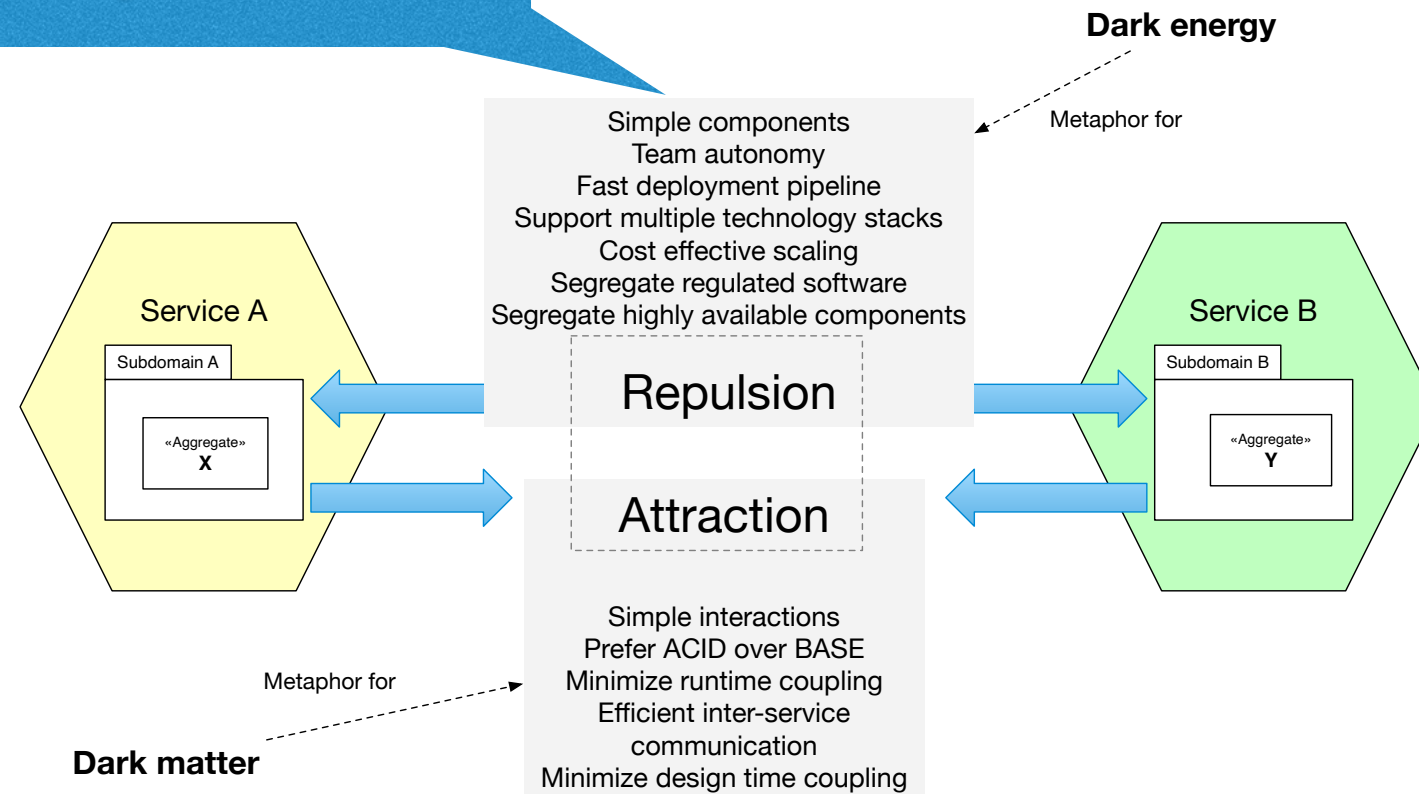


Queries



Dark energy/dark matter forces drive design decisions

Assigning subdomain to existing service potentially violates



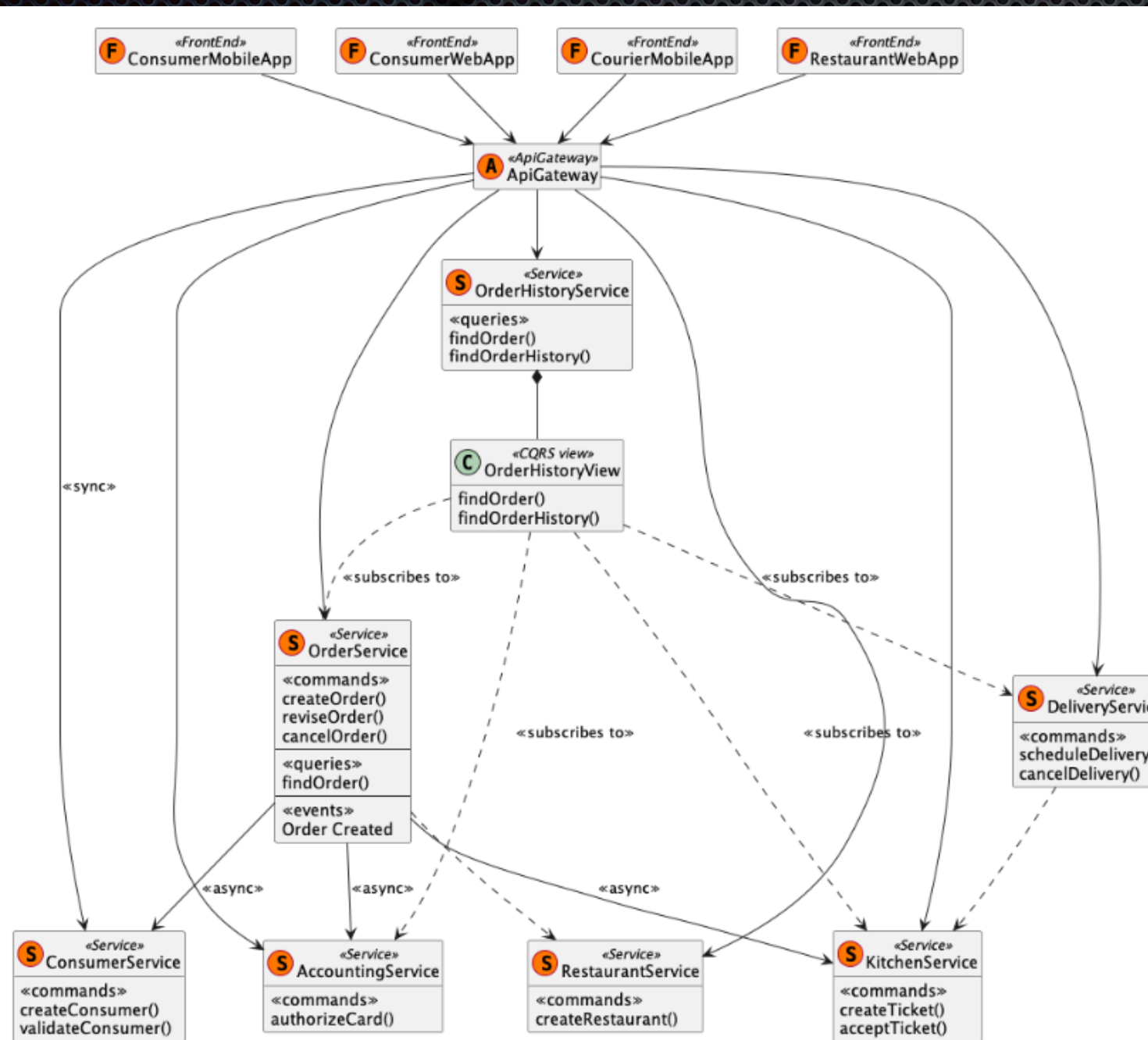
Collaboration design potentially violates

Any assignment resulting in a distributed operation potentially violates

Agenda

- Overview of designing services and system operations
- Documenting services and collaborations
 - Ranking operations
 - Generating candidate service and operation designs
 - Evaluating candidate designs

Documenting services: overview



The responsibilities of each service are as follows:

- **Consumer Service** - manages consumers
- **Restaurant Service** - manages restaurants, e.g. menus
- **Courier Service** - manages couriers
- **Order Service** - manages orders

Documenting services: microservice canvas

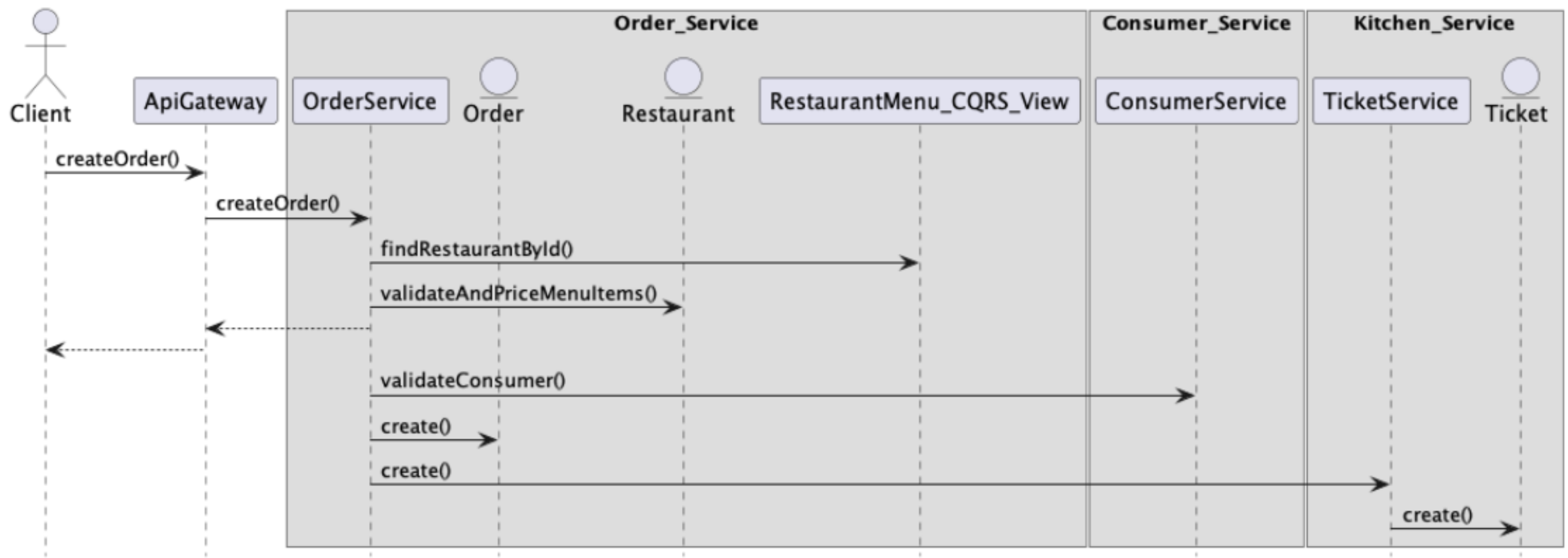
Service canvas		
Name:	Order Service	
Description:		
Responsibilities	• Manages orders	
Subdomains	• Order management	
Service API		
Commands	Queries	Events Published
• createOrder()	• findOrder()	• OrderCreated
Non-functional requirements		
Dependencies		
Invokes	Subscribes to	
	Restaurant Service: • Restaurant Created • Restaurant Updated • Restaurant ...	

<https://github.com/cer/microservice-canvas>

@crichardson

Documentation: sequence diagrams showing collaboration

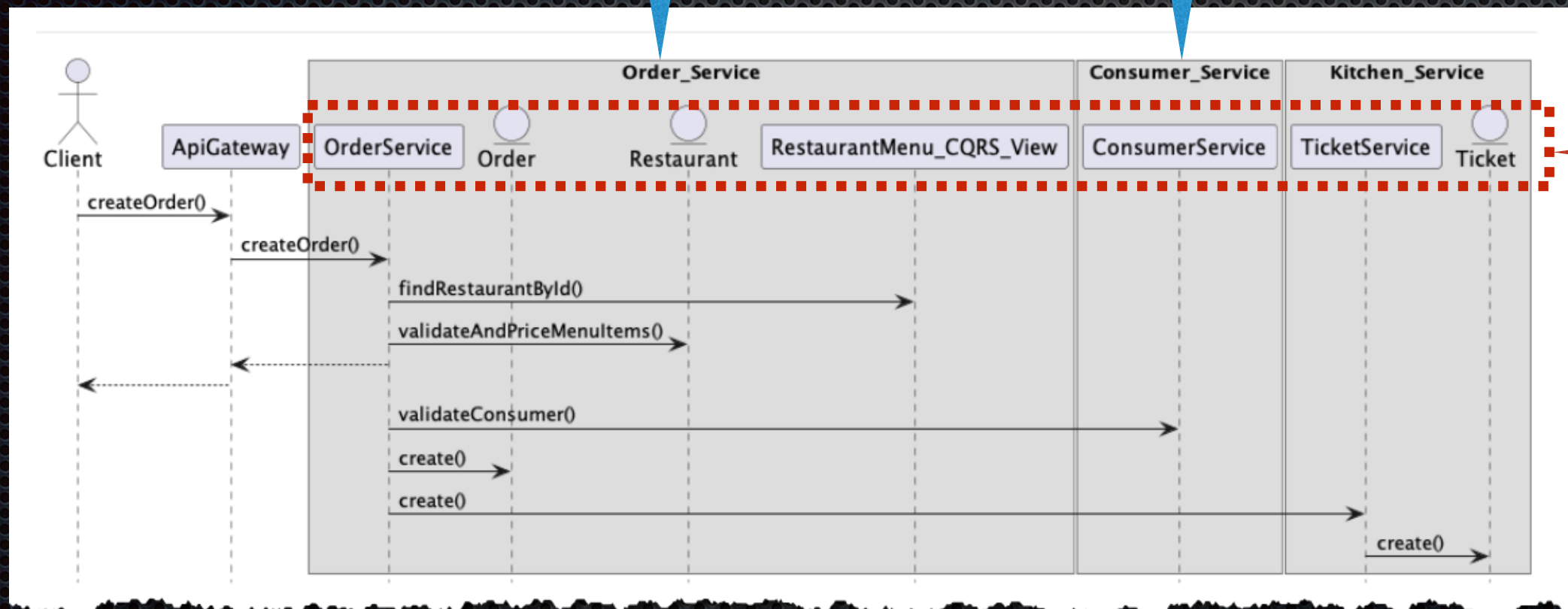
Per-operation/scenario



About the sequence diagrams

(Micro)service

(Micro)service



Classes
within
services

Documenting system commands: design canvas

System command design canvas	
Name:	createOrder()
Entry point:	
Service:	Order Service
Operation:	createOrder()
Collaboration patterns:	
Sagas	• Create Order
Command-side Replicas	• Restaurant Menu View

Documenting system operations: query design canvas

System query design canvas	
Name:	findOrder()
Entry point:	
Service:	
Operation:	
Collaboration patterns:	
API Composition	
CQRS	

Agenda

- Overview of designing services and system operations
- Documenting services and collaborations
- Ranking operations
 - Generating candidate service and operation designs
 - Evaluating candidate designs

Ranking system operations

- Architecture = trade-offs = some operations will have a better design than others
- Higher ranked operations
 - more freedom to partition subdomains
 - have a bigger impact on the architecture
- Lower ranking operations
 - Less freedom since many subdomains already partitioned
 - Less able to influence architecture

System operation ranking criteria

- High business criticality - important to get these system operations right
- High technical risk - more challenging = need fewer constraints
 - Complexity
 - Large data volumes
- ...

What's criteria are available

- System operation
 - SLO
 - Volume of data
 - General complexity # subdomains
- System operation's subdomains
 - Type: Core vs....
 - SLO
 - Complexity
 - Regulatory

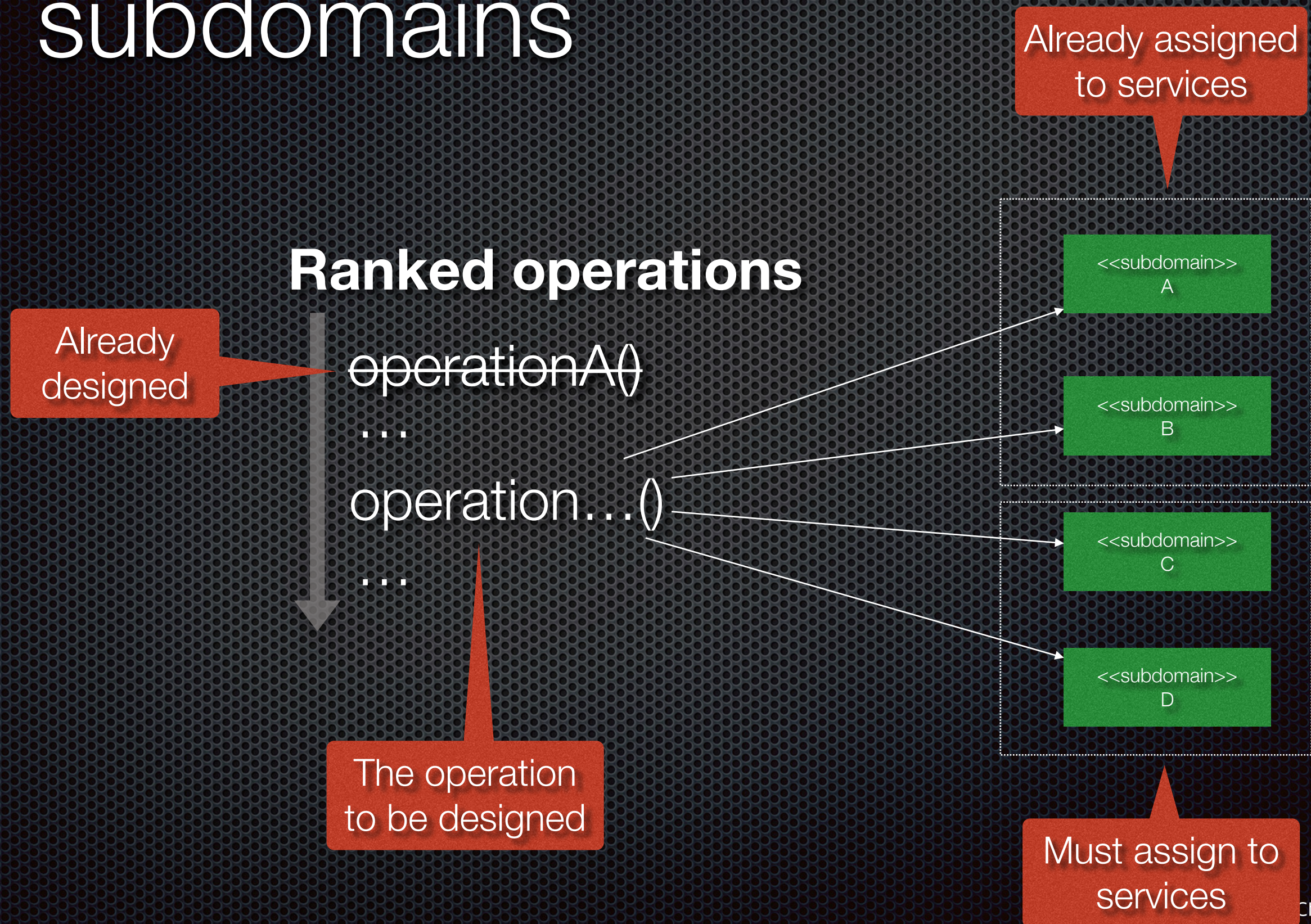
Examples rankings....

Application	Important	Less important
FTGO	Order flow: createOrder(), acceptTicket(), noteCourier*(), ...	findOrderHistory()
Credit card processor	authorizeCard(), captureCard(), ...	registerMerchant()
Email marketing	sendScheduledEmails()	registerAccount()

Agenda

- Overview of designing services and system operations
- Documenting services and collaborations
- Ranking operations
- Generating candidate service and operation designs
- Evaluating candidate designs

Grouping an operation's subdomains



Group an operation's subdomains...

Blank sheet!!!!

Ranked operations

createOrder()

acceptTicket(ticketID)

??? Service

Order
Management

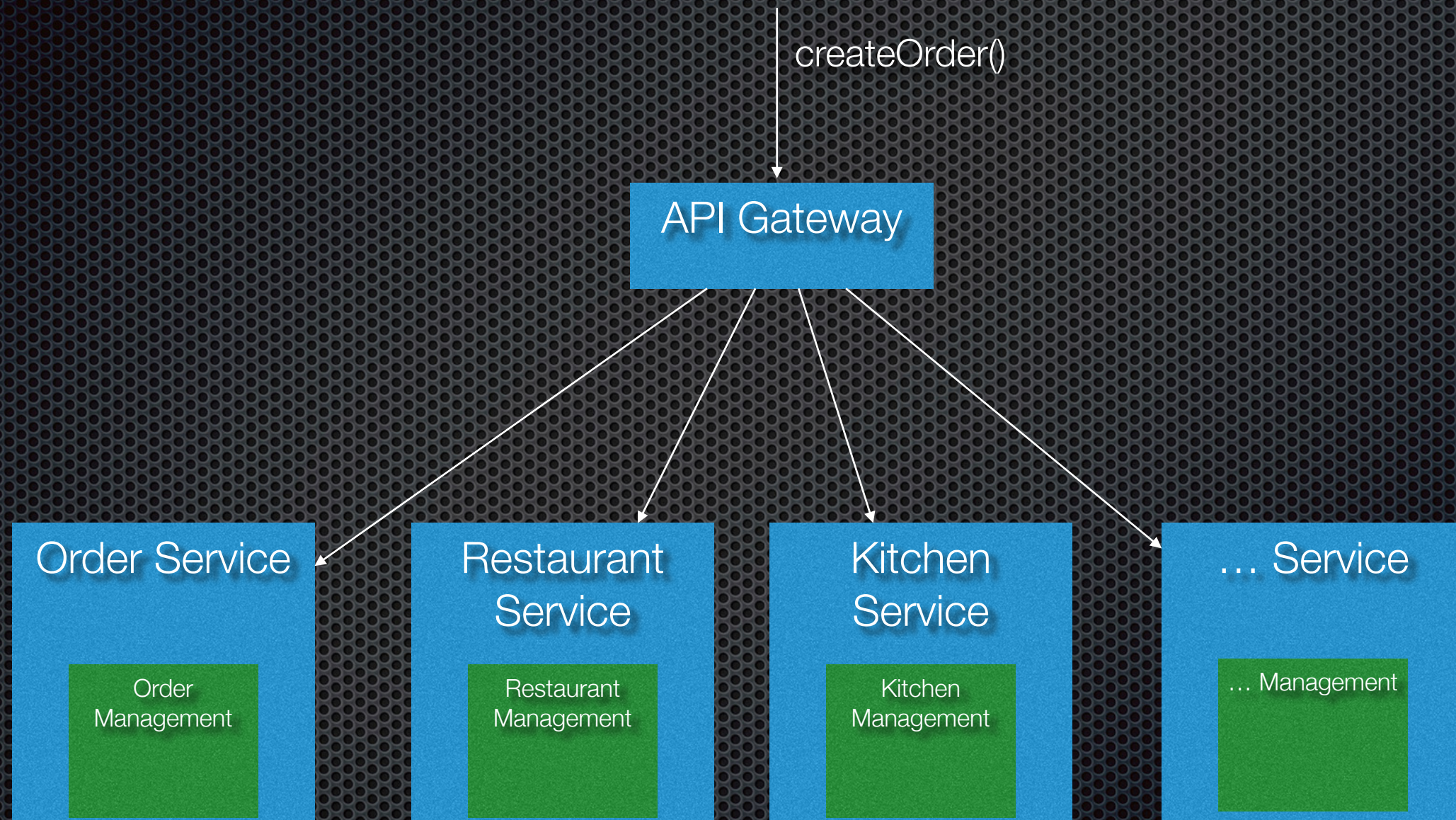
Kitchen
Management

Restaurant
Management

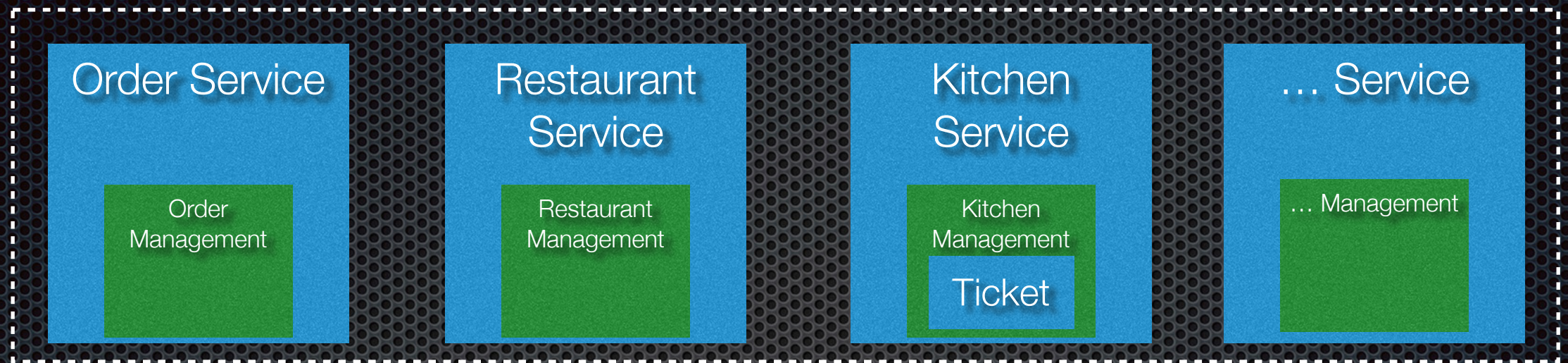
... Management

Must assign to services

Architecture: after designing createOrder()



...Group an operation's subdomains...



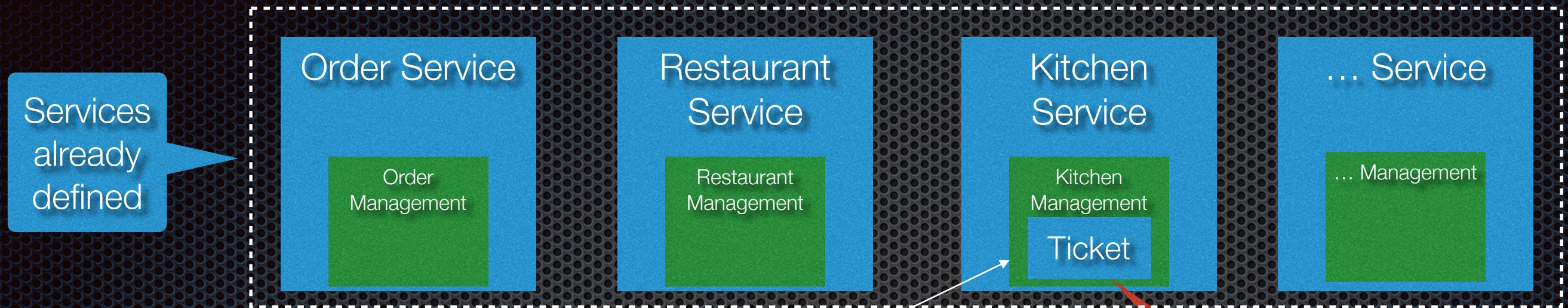
Ranked operations

`createOrder()`

`acceptTicket(ticketID)`



..Group an operation's subdomains...



Ranked operations

`createOrder()`

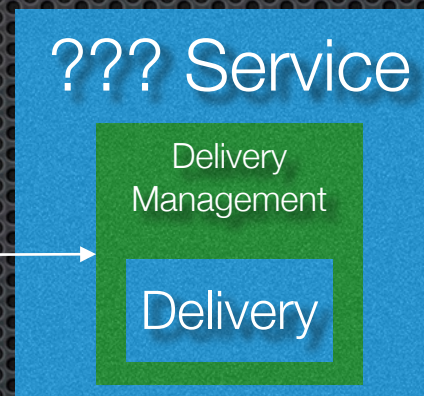
`acceptTicket(ticketID)`

Pre-conditions:
Exists **Ticket** t
 where t.ID = ticketID
 and t.state = PENDING

Post-conditions:
t.state became ACCEPTED
Delivery d was created
d.state became SCHEDULED
d... Became ...

`noteAccepted()`

`scheduleDelivery()`



Already assigned

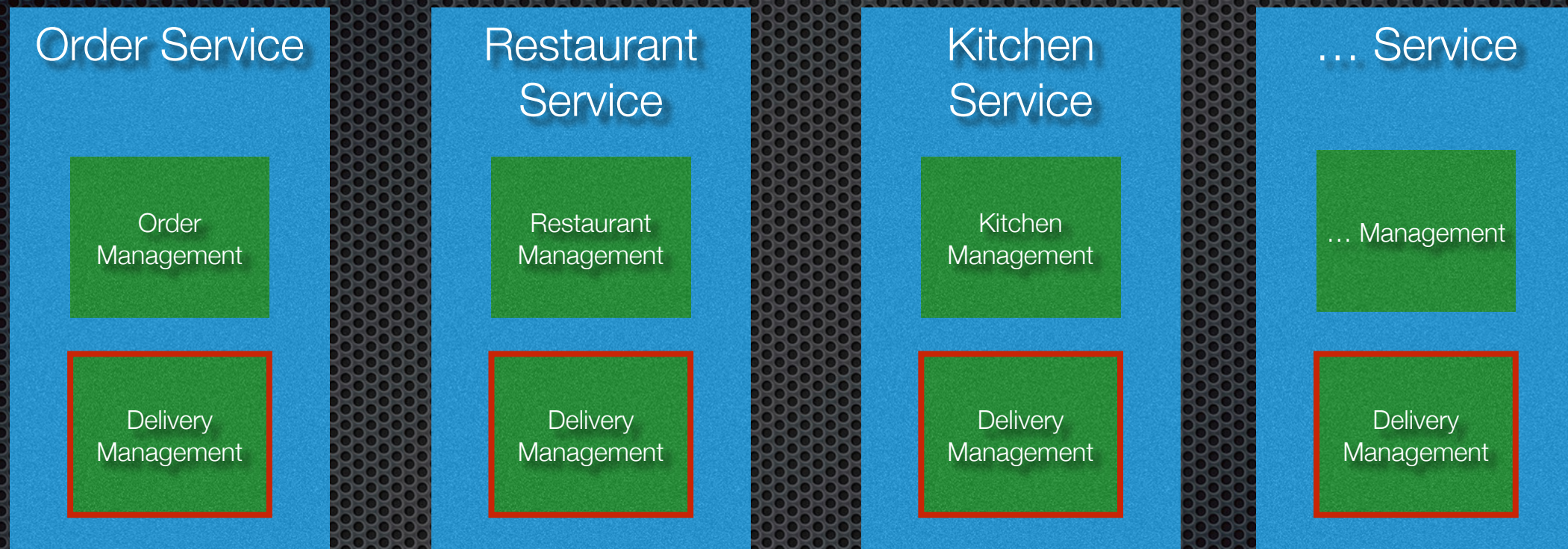
Must assign to either
existing service or new
service

Many possible designs

- For each possible assignment of subdomains to existing/new services:
 - For each applicable combination of pattern(s)
 - For each way of applying the pattern(s)
 - ...

Multiple possible locations for Delivery Management

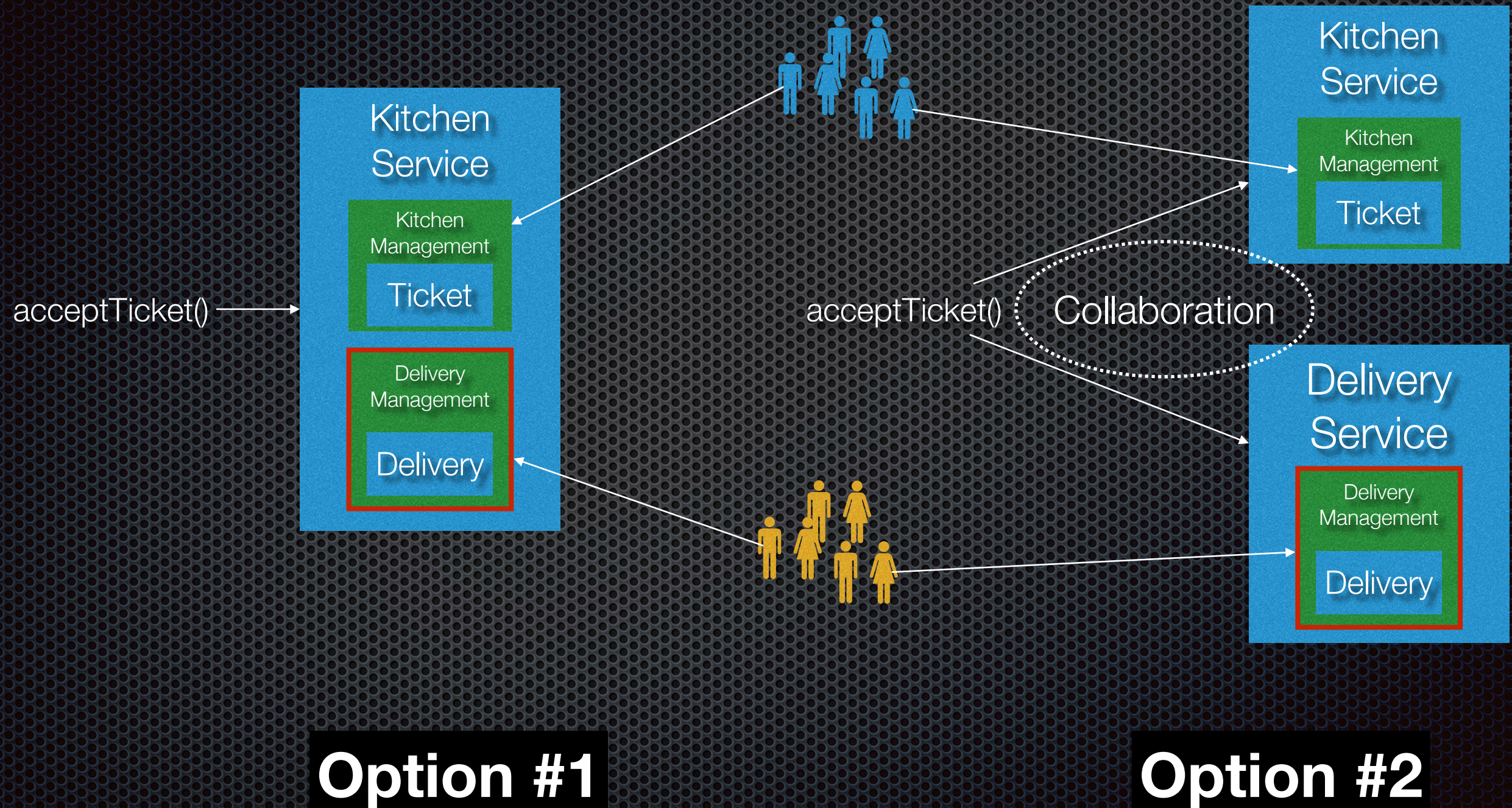
Existing



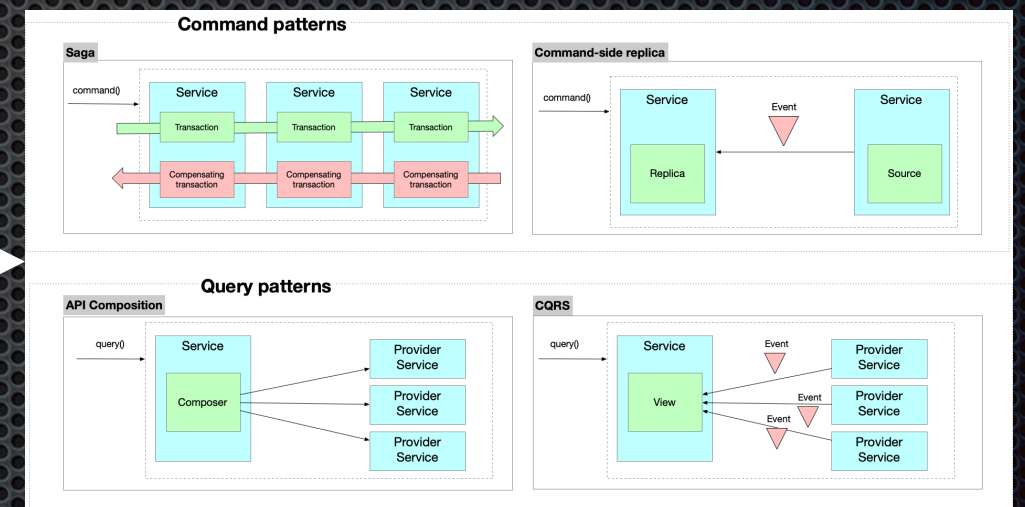
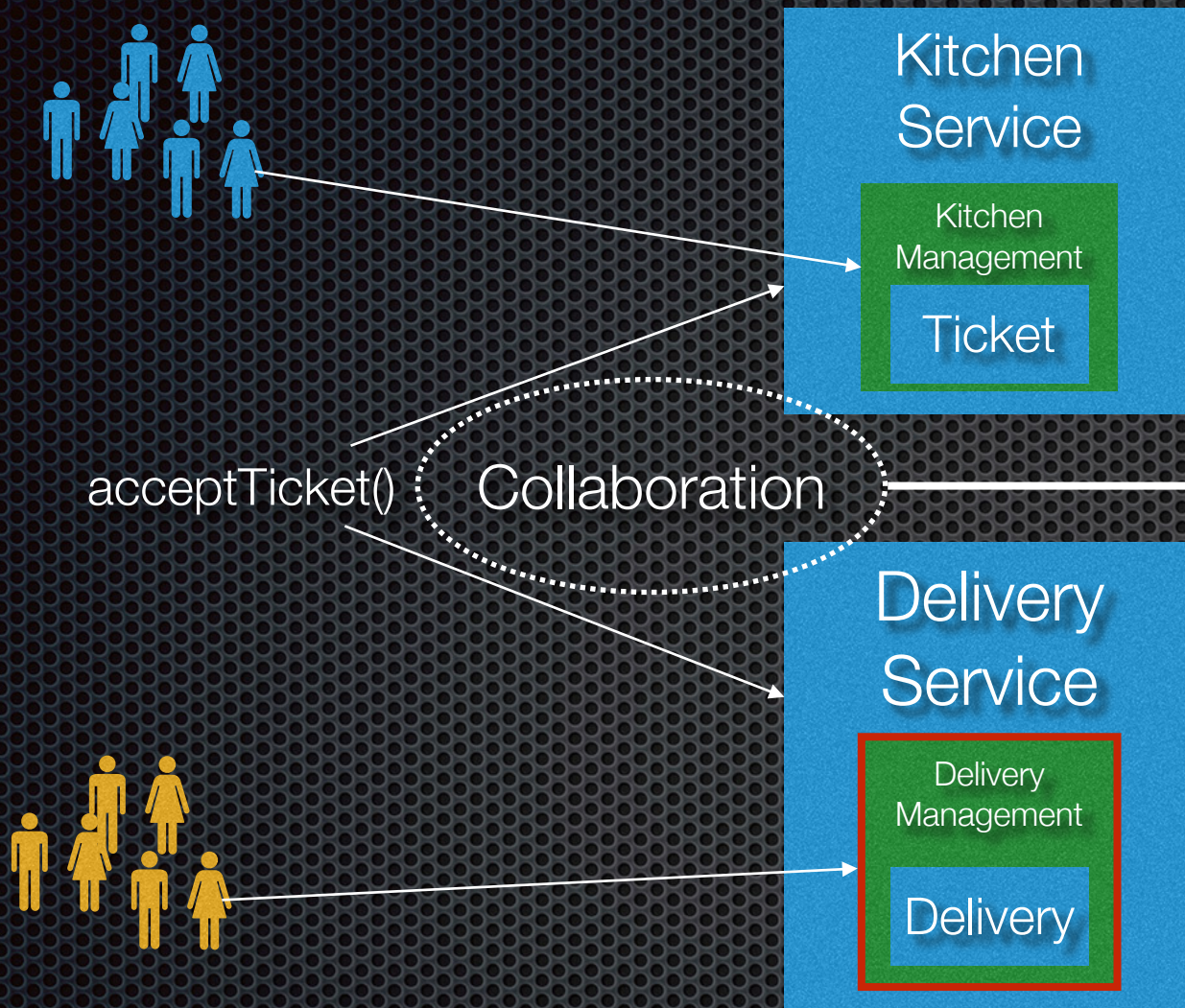
New



Two possible assignments



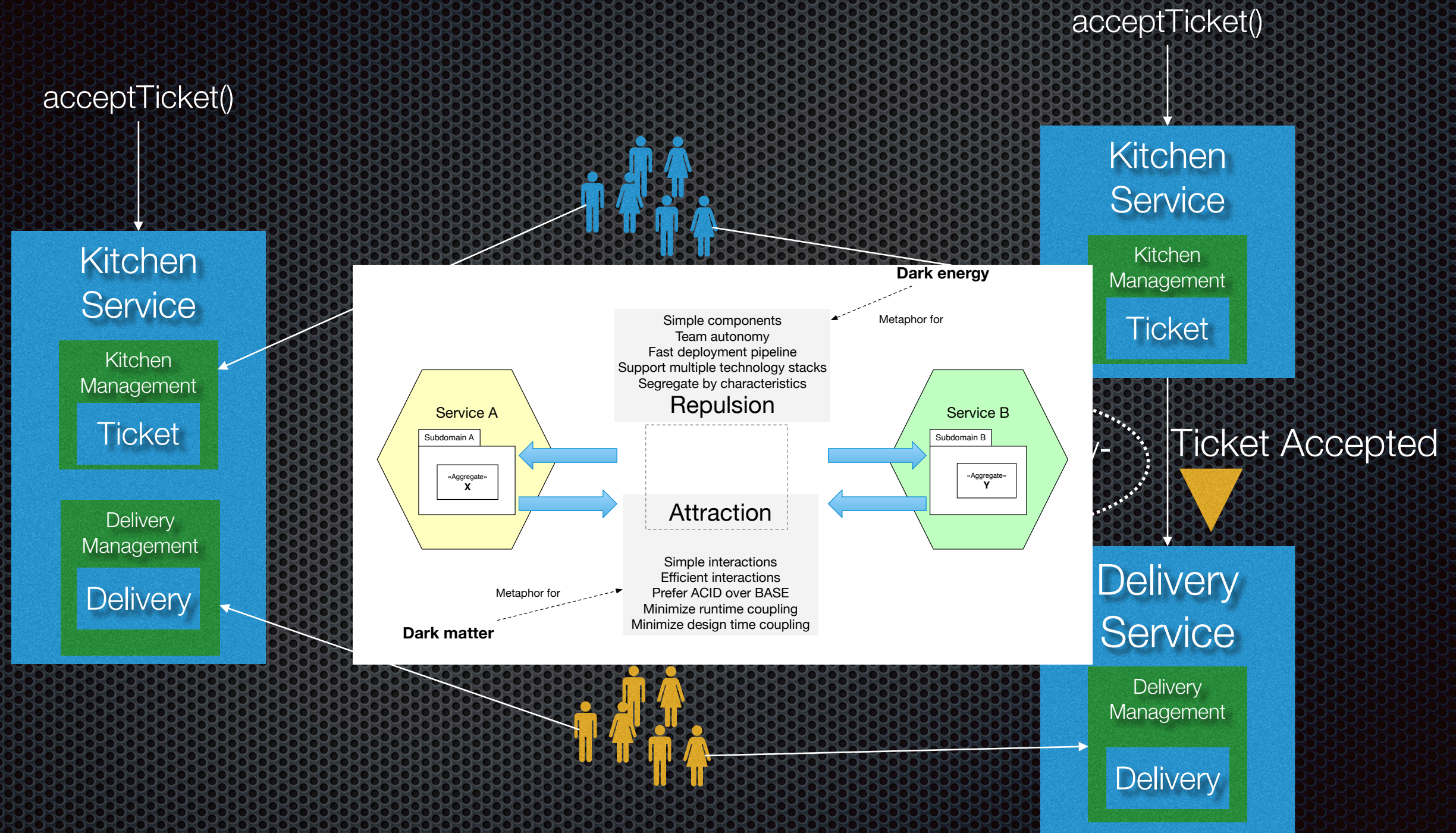
Option #2: multiple ways to design collaboration



Agenda

- Overview of designing services and system operations
- Documenting services and collaborations
- Ranking operations
- Generating candidate service and operation designs
- Evaluating candidate designs

Evaluate using dark energy and dark matter



Option #1

Option #2

Evaluate alternative designs using dark energy and dark matter forces

Within Kitchen Service

Separate Delivery Service

Dark energy, repulsive forces

Simple components

Team autonomy

Fast deployment pipeline

Support multiple technology stacks

Segregate by characteristics

?

?

Dark matter, attractive forces

Simple interactions

Efficient interactions

Prefer ACID over BASE

Minimize runtime coupling

Minimize design-time coupling

?

?

Dark energy repulsive forces $\Rightarrow$ subdomains in different services



Dark energy forces: reasons to create a Delivery Service



But do they apply in this situation?

Does defining a Delivery service **solve** a problem?

Evaluating: Simpler services

“Everything should be made
as simple as possible.
But not simpler.”



Kitchen Service

<<subdomain>>
Kitchen
Management

<<subdomain>>
Delivery
Management

Complicated

dependencies

versus

Complex

Kitchen
Service

<<subdomain>>
Kitchen
Management



Albert Einstein

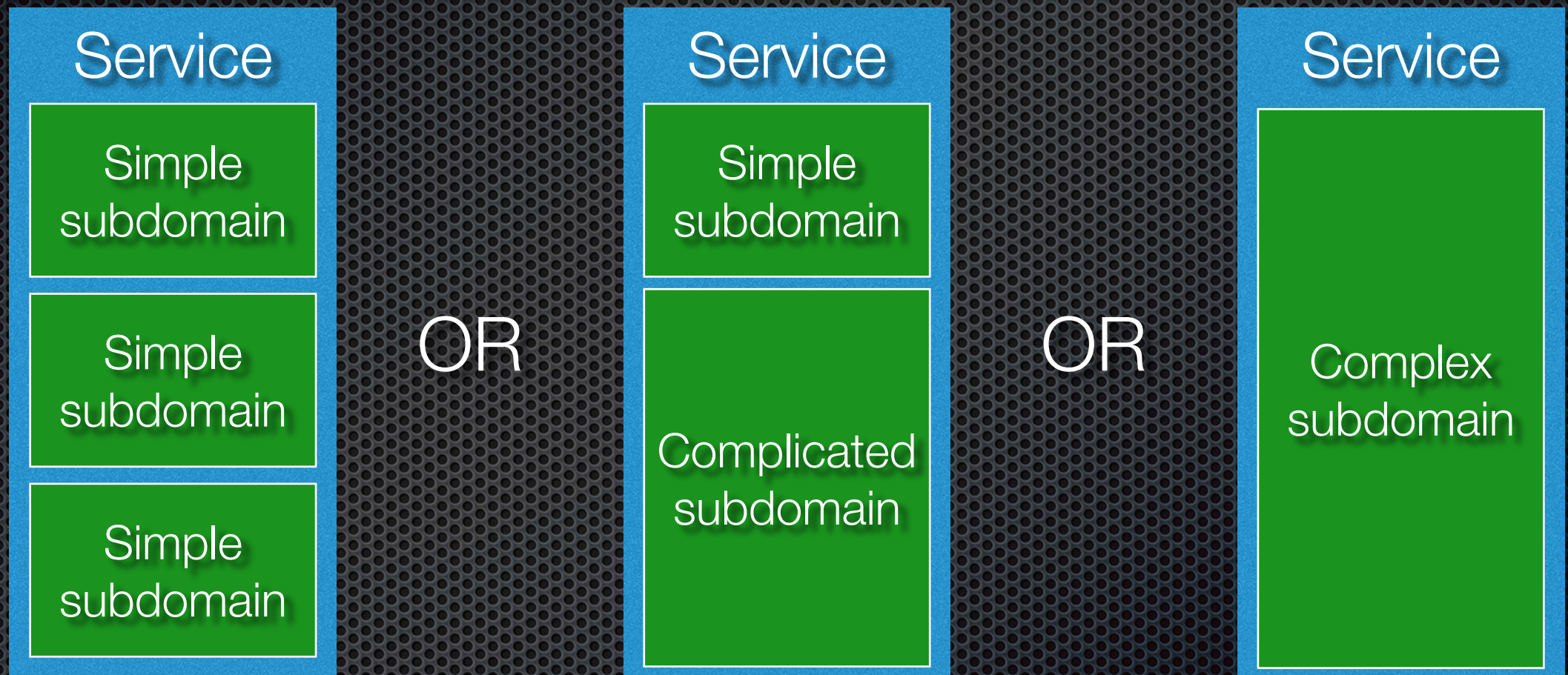
Delivery
Service

<<subdomain>>
Delivery
Management

More complex service

Simpler services: easier to
understand, develop, test, ...

Evaluating service complexity: $\text{sumOf}(\text{complexity}(\text{subdomains}))$



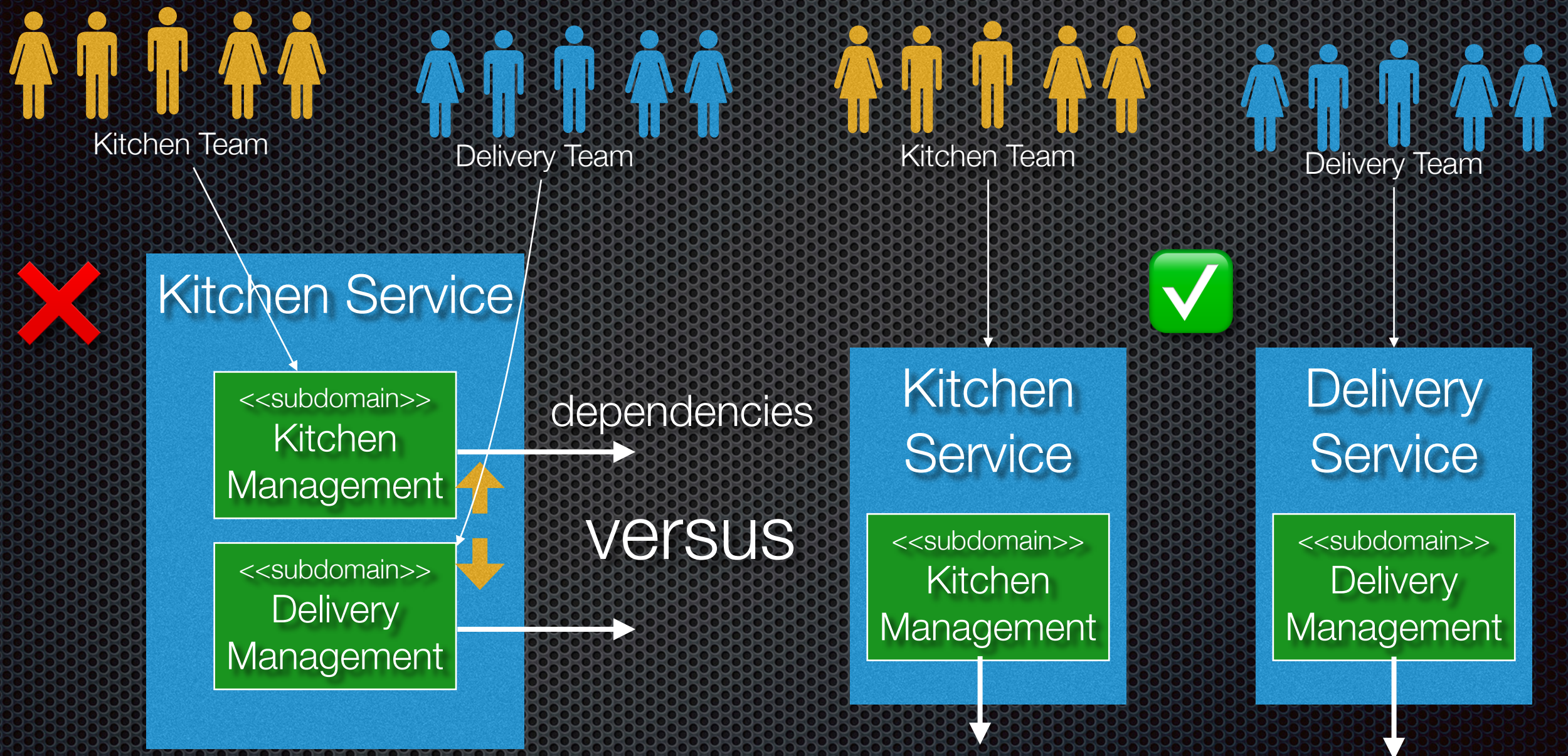
Especially

Architectural smell:

Bloated service

Justify overly “bloated” services

Evaluating: Team autonomy



Coordination required
Contention for resources

Develop, push, build, test
and deploy independently

Evaluating: Fast deployment

Kitchen Service

<<subdomain>>
Kitchen
Management

<<subdomain>>
Delivery
Management

Deployment pipeline

Longer lead time



More complex build*

* Parallelizing building/testing partially helps

VS.

Kitchen
Service

<<subdomain>>
Kitchen
Management

Deployment pipeline

Shorter lead time



Delivery
Service

<<subdomain>>
Delivery
Management

Deployment pipeline

Simpler build

Evaluating: Support multiple technology stacks

- Does Delivery Management use the same technology stack as Kitchen Management?
 - Same language, framework
 - Compatible transitive dependencies
- Does Delivery Management introduce new dependencies that would complicate technology upgrades?
 - Service upgrade effort proportional # dependencies
 - A dependency might not support newer versions of libraries, JDK, etc



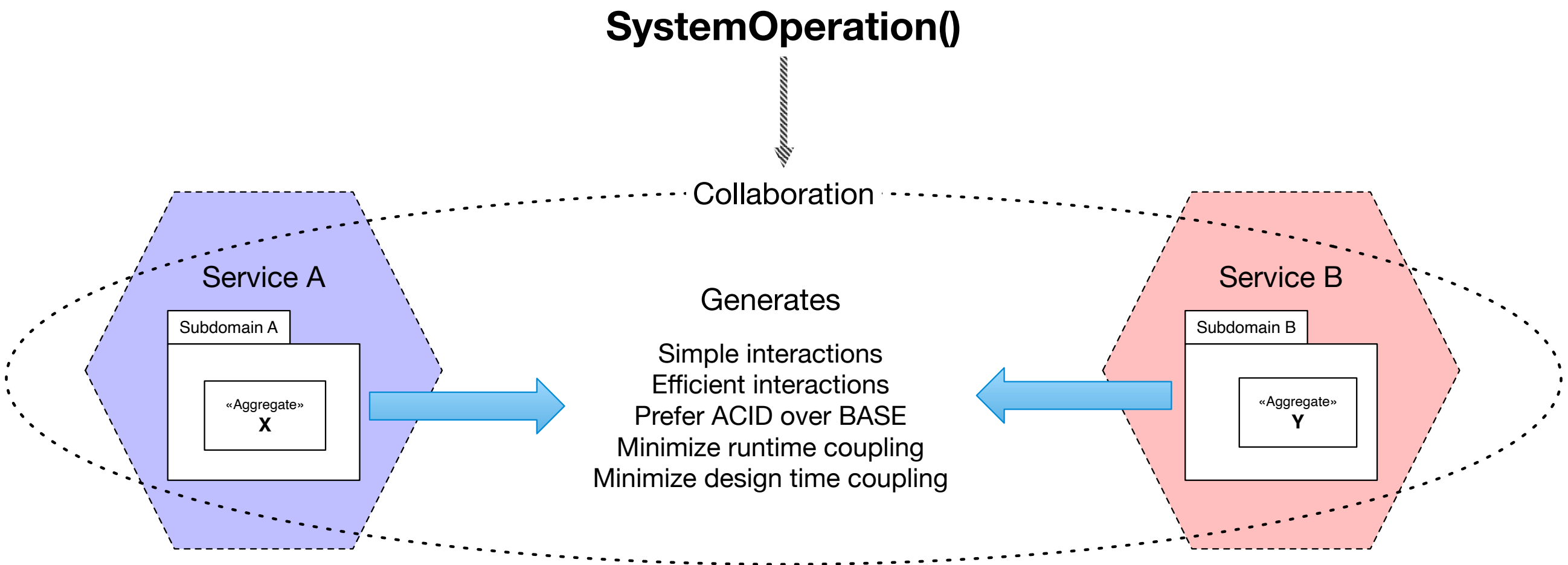
Evaluating: Separate subdomains by characteristics

Subdomain characteristic	Kitchen management	Delivery Management
Resource requirements	Similar	
Regulations, e.g. SaMD/PCI/...	None	None
Business criticality/tier	Tier 1	Tier 1
Security, e.g. PII, ...	Low	Low
DDD core/supporting/generic	Core	Core

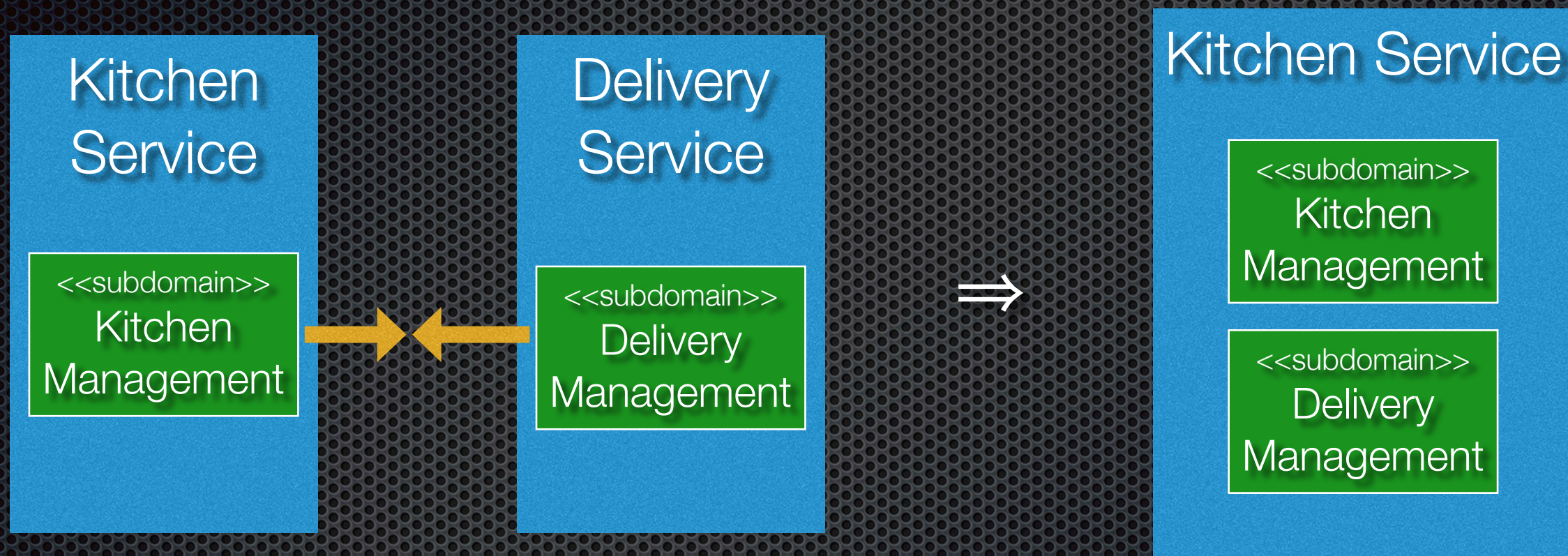
Summary: designing acceptTicket()

	Within Kitchen Service	Delivery Service
Dark energy, repulsive forces		
Simple components	✗	✓
Team autonomy	✗	✓
Fast deployment pipeline	✗	✓
Support multiple technology stacks		N/A
Segregate by characteristics		N/A
Dark matter, attractive forces		
Simple interactions		
Efficient interactions		
Prefer ACID over BASE	?	?
Minimize runtime coupling		
Minimize design-time coupling		

Dark matter attractive forces $\Rightarrow$ subdomains in same service



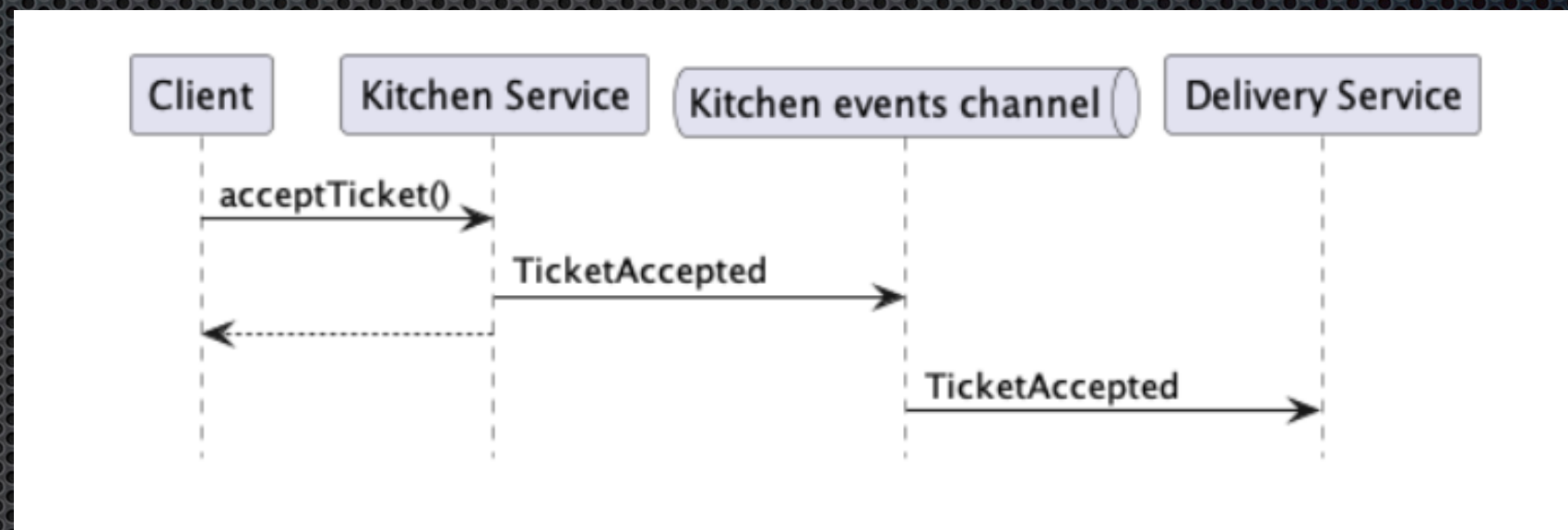
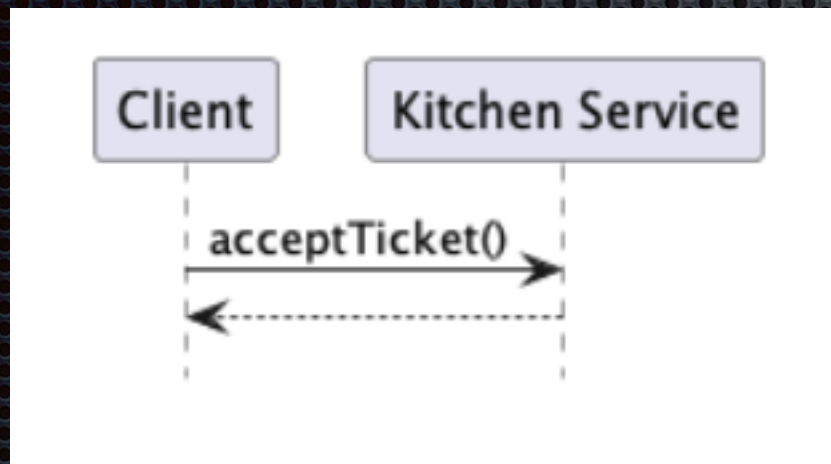
Dark matter forces: reasons to colocate Kitchen and Delivery management



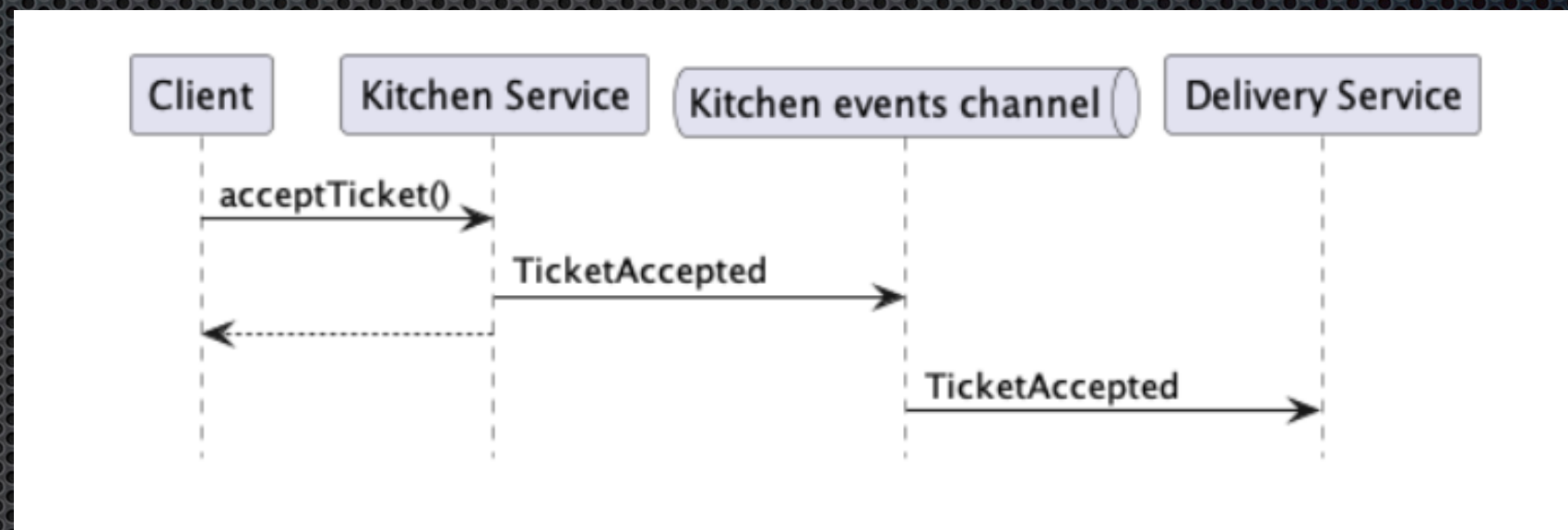
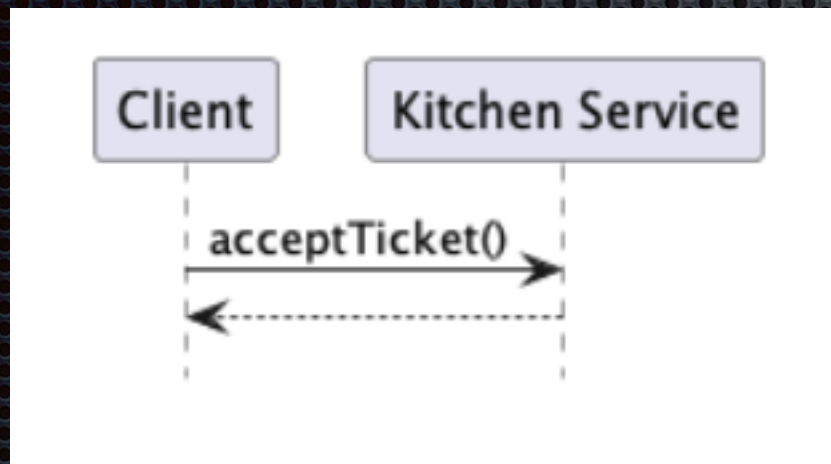
But do they apply in this situation?

Does defining a Delivery Service **create** a problem?

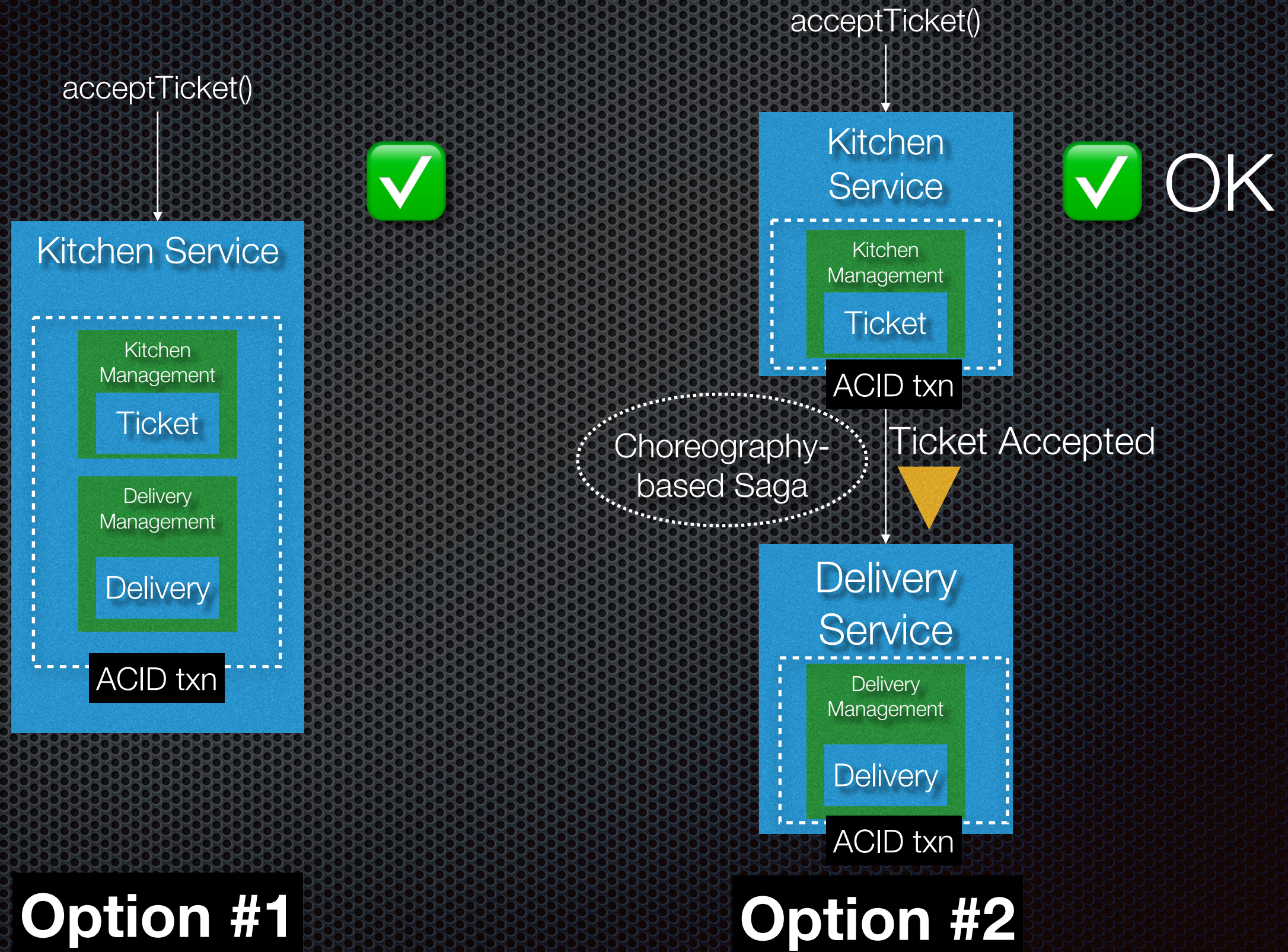
Evaluating: Simple interactions



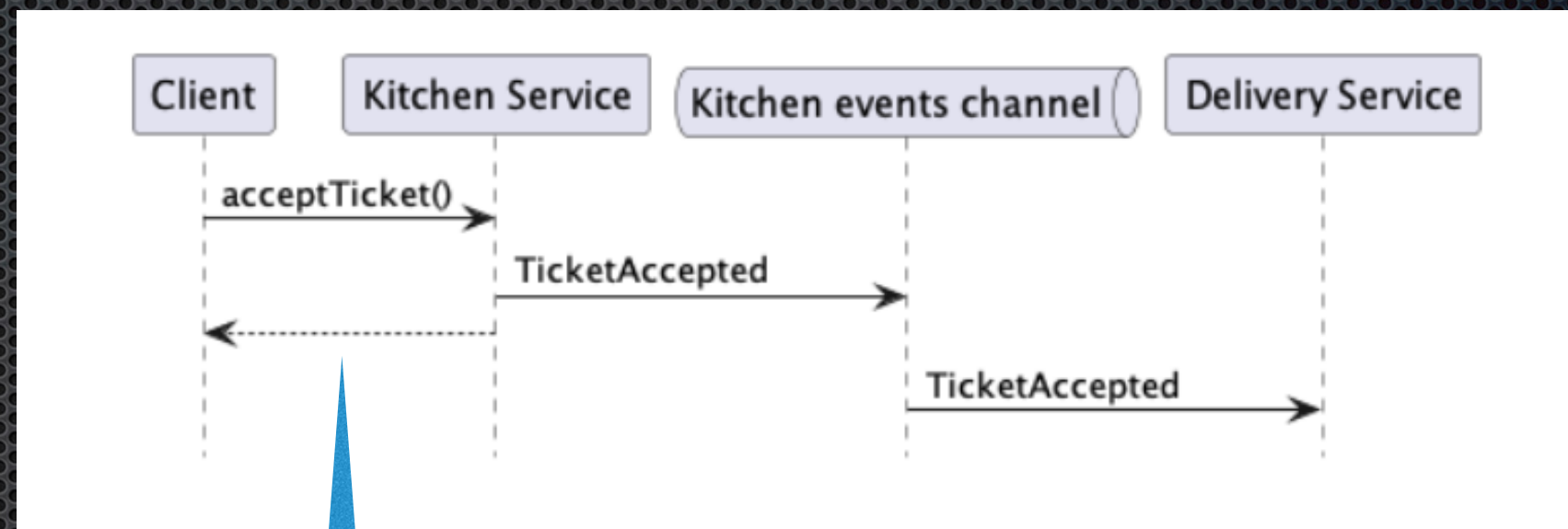
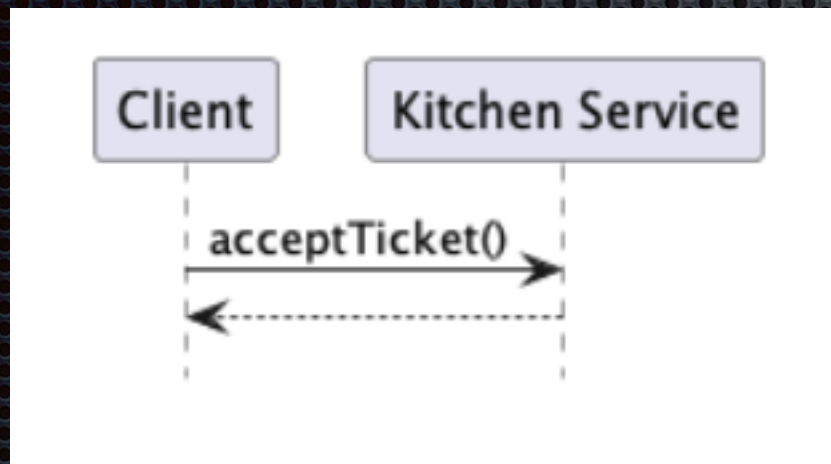
Evaluating: Efficient interactions



Evaluating: Prefer ACID over BASE

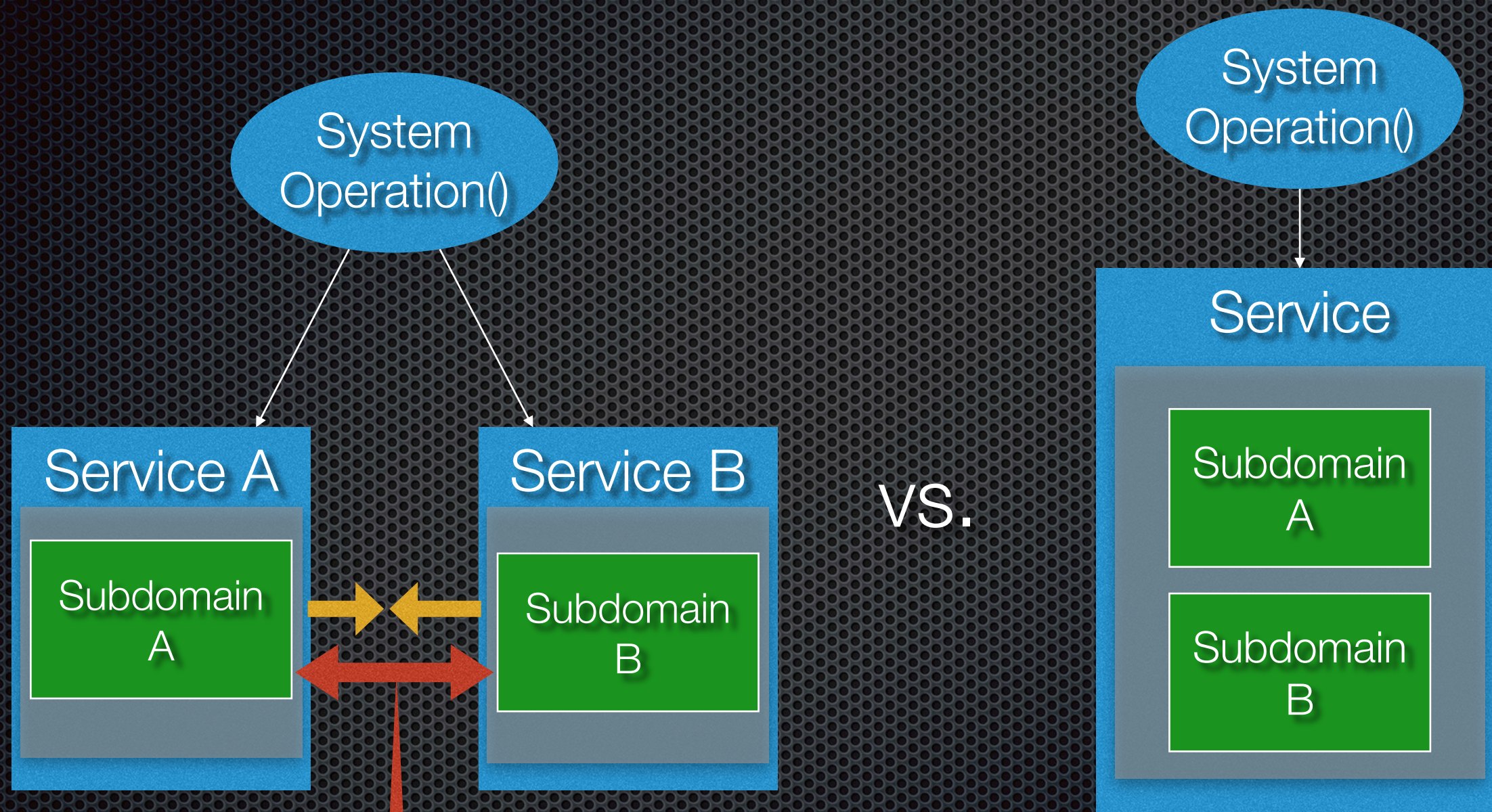


Evaluating: Minimize runtime coupling



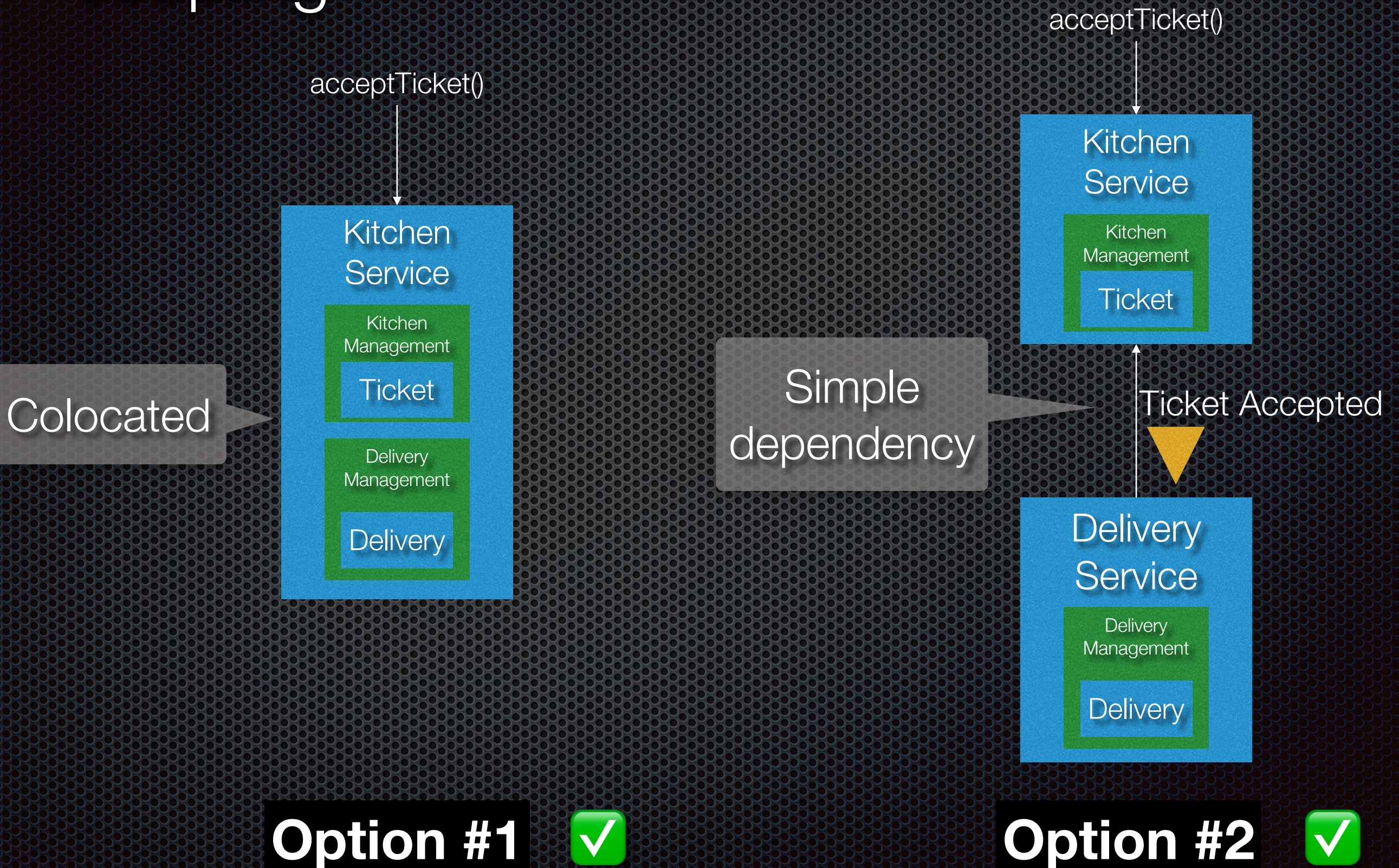
OK unless response must contain delivery time?

Evaluating: Minimize design-time coupling

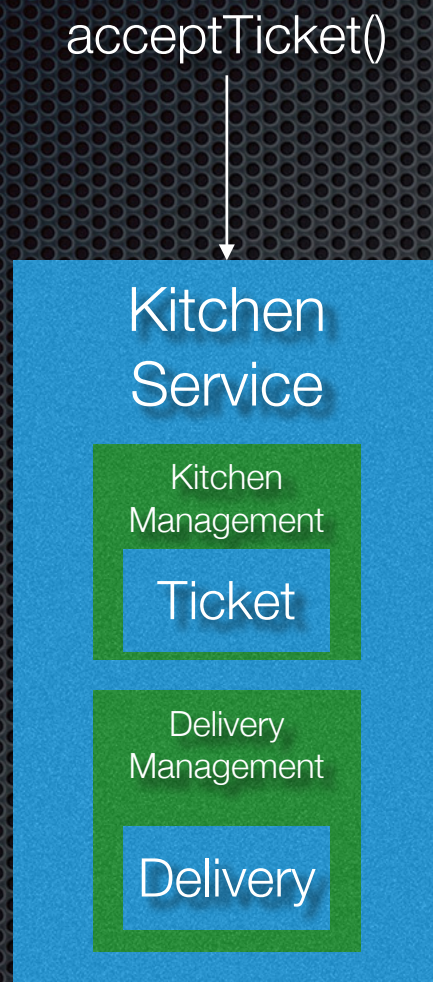


Tight coupling => expensive lock step changes

Evaluating: Minimizing design-time coupling



Option: Kitchen Service



Dark matter, attractive forces

Simple interactions

Efficient interactions

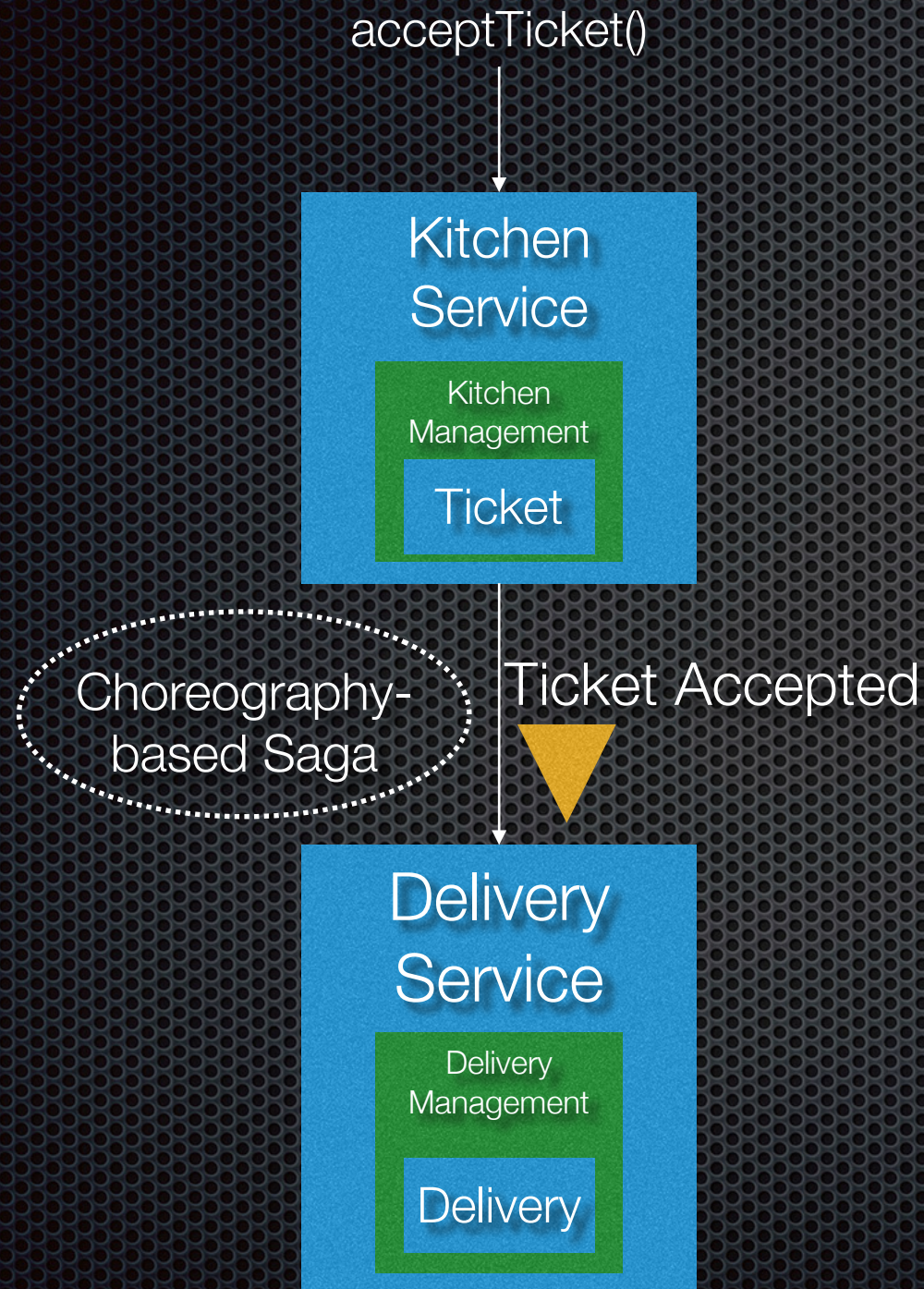
Prefer ACID over BASE

Minimize runtime coupling

Minimize design-time coupling



Option: Delivery Service



Dark matter, attractive forces

Simple interactions



Efficient interactions



Prefer ACID over BASE



Minimize runtime coupling



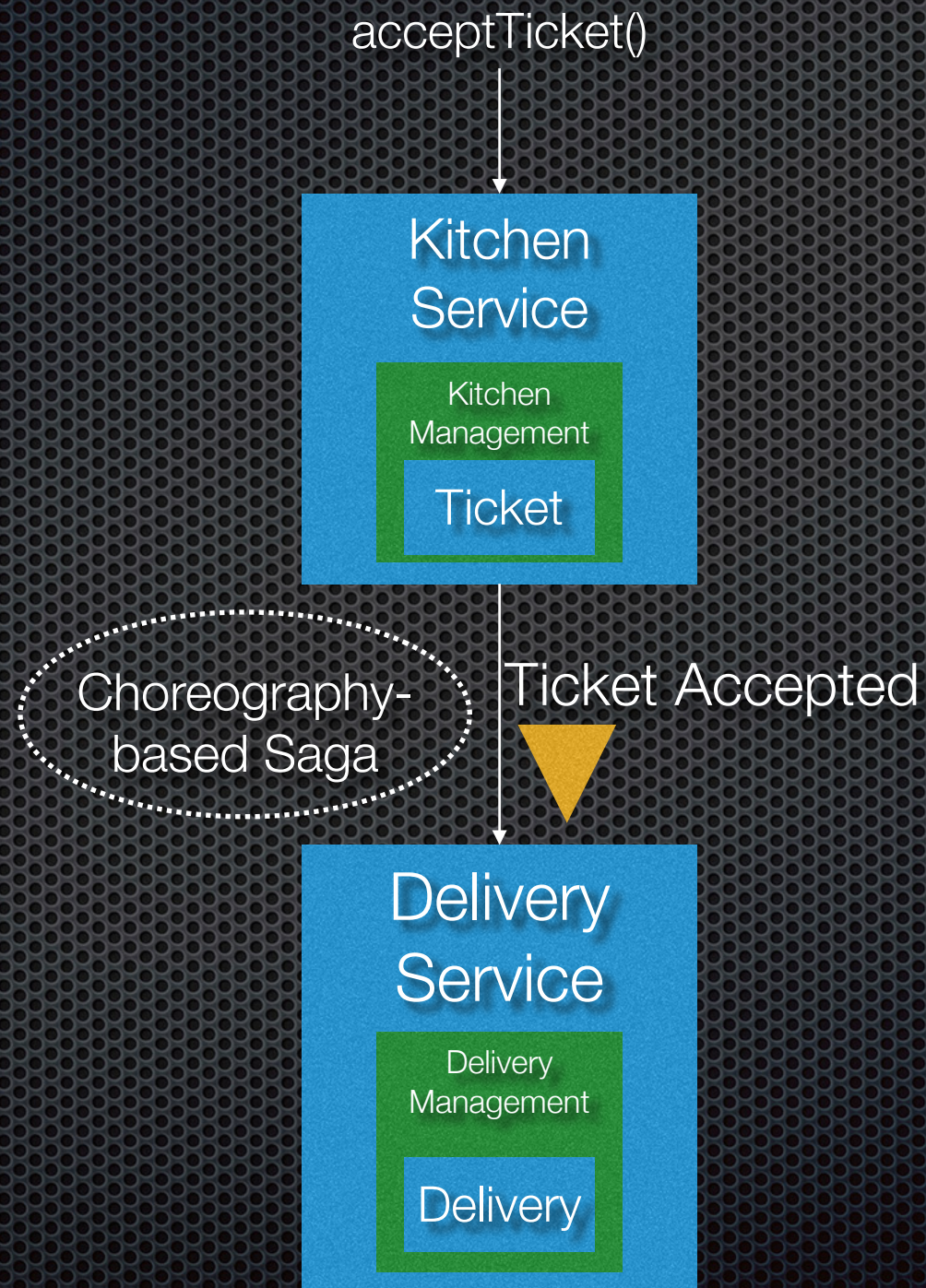
Minimize design-time coupling



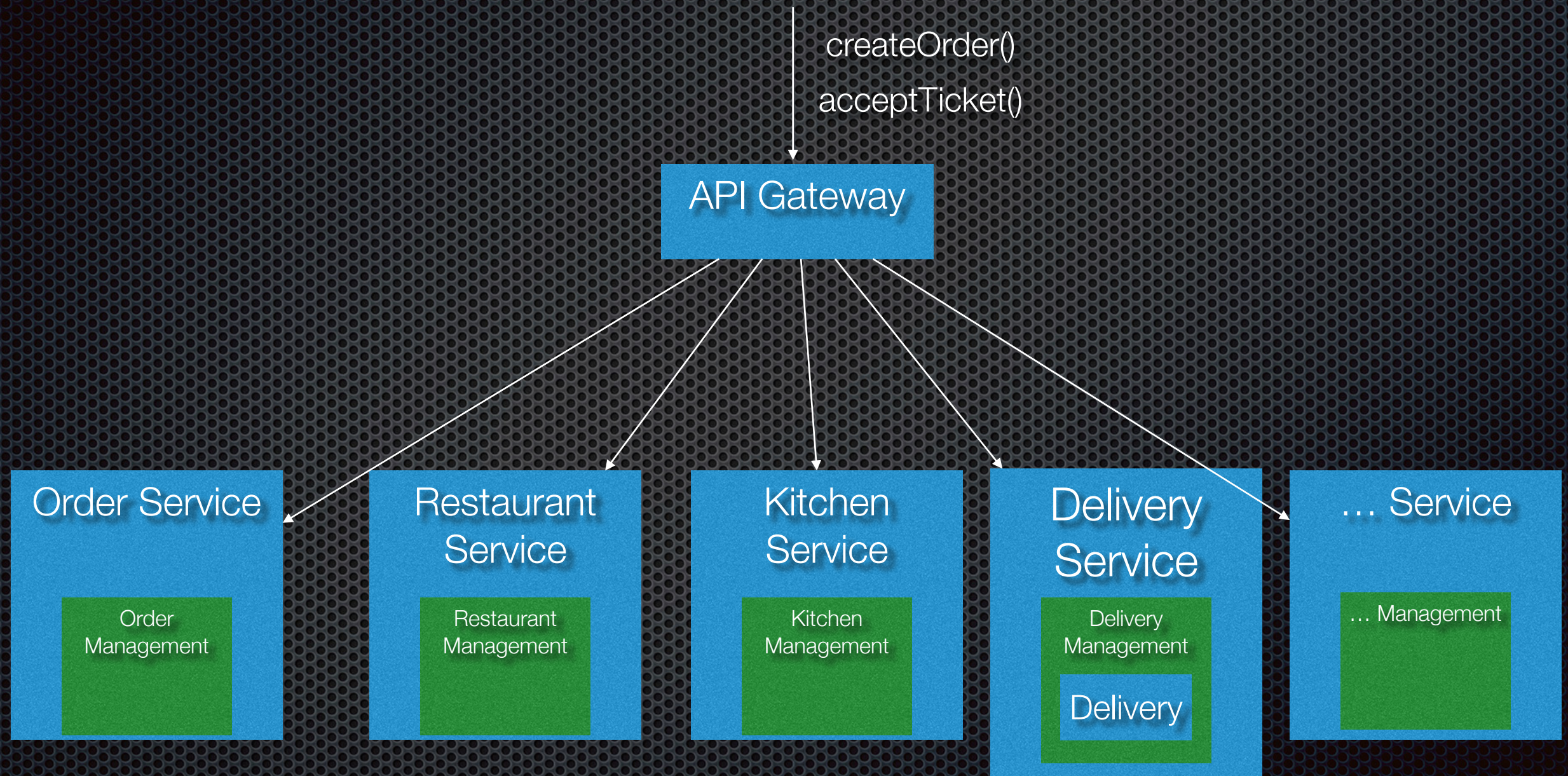
Summary: designing Kitchen Management

	Within Kitchen Service	Delivery Service
Dark energy, repulsive forces		
Simple components	✗	✓
Team autonomy	✗	✓
Fast deployment pipeline	✗	✓
Support multiple technology stacks		N/A
Segregate by characteristics		N/A
Dark matter, attractive forces		
Simple interactions		✓
Efficient interactions		✓
Prefer ACID over BASE	✓	✓ OK
Minimize runtime coupling		✓
Minimize design-time coupling		✓

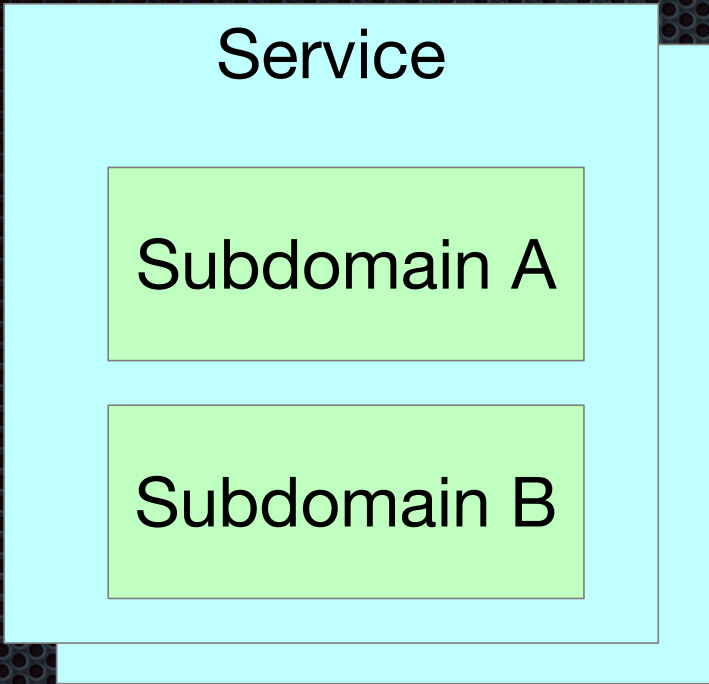
Implementing acceptTicket()



Architecture: after designing acceptTicket()

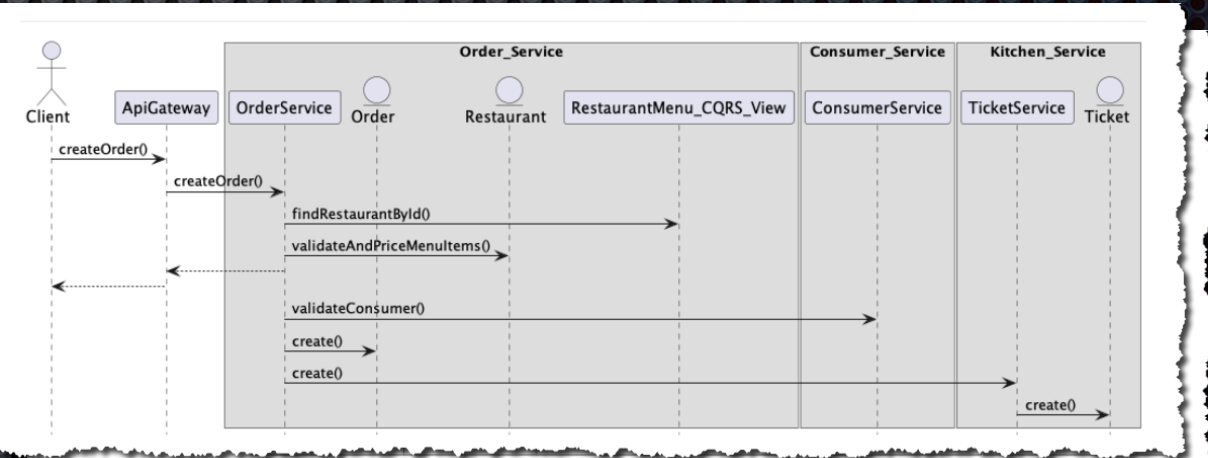


Summary: Deliverables



System command design canvas	
Name:	createOrder()
Entry point:	
Service:	Order Service
Operation:	createOrder()
Collaboration patterns:	
Sagas	• Create Order
Command-side Replicas	• Restaurant Menu View

Service canvas		
Name:	Order Service	
Description:		
Responsibilities	• Manages orders	
Subdomains	• Order management	
Service API		
Commands	Queries	Events Published
• createOrder()	• findOrder()	• OrderCreated
Non-functional requirements		
Dependencies		
Invokes	Subscribes to	
	Restaurant Service: • Restaurant Created • Restaurant Updated • Restaurant ...	



- Templates:
- 03-01-ranking-system-operations
 - 03-02-designing-operations-and-services

 [@crichardson](https://twitter.com/crichardson) chris@chrisrichardson.net

Questions?

adopt.microservices.io